

Sistema di Noleggio Mezzi

Corsi di Laurea in Ingegneria Informatica

LM32 – A.A. 2025/2026

INGEGNERIA DEL SOFTWARE

Studenti:

Anastasia Alida Salamanca

Lorenzo Pantano

Sara Scavone

INDICE

Sommario

INDICE.....	2
PREFAZIONE	3
FASE DI IDEAZIONE E ANALISI DEI REQUISITI.....	3
MODELLO DEI CASI D'USO.....	4
GLOSSARIO	13
LOGICA DI BUSINESS E REGOLE DI DOMINIO	14
ANALISI ORIENTATA AGLI OGGETTI	16
ITERAZIONE 1	16
UC3: Visualizza Disponibilità Mezzi.....	16
UC4: Avvia Noleggio.....	19
UC5: Concludi Noleggio	21
ITERAZIONE 2	24
UC1: Autentica Cassiere	24
UC2: Registrazione Cliente	26
UC7: Aggiungi Nuovo Mezzo	28
ITERAZIONE 3	31
UC6: Segnala Guasto	31
UC8: Effettua Manutenzione.....	34
UC9: Effettua Prenotazione WEB	36
UC10: Gestisci Blacklist	39
DESIGN PATTERN UTILIZZATI.....	41
INTERFACCIA UTENTE WEB.....	43
TESTING	44

PREFAZIONE

L'obiettivo principale del progetto è stato quello di applicare i principi e le metodologie dell'Ingegneria del Software per risolvere un problema reale e complesso di dominio applicativo: la gestione operativa e telematica di una flotta di veicoli, coordinando in modo sicuro ed efficiente le interazioni tra la clientela e il personale autorizzato (staff).

Durante l'intero ciclo di vita dello sviluppo, si è posto un forte accento non limitato alla mera stesura del codice, ma alla corretta ingegnerizzazione del prodotto. A tale scopo, sono stati applicati rigorosamente i paradigmi della Programmazione Orientata agli Oggetti (OOP) e i principali Design Pattern (GoF) per garantire un'architettura modulare, manutenibile e scalabile. Inoltre, è stata implementata una severa strategia di Unit e Integration Testing per certificare la solidità e la correttezza della Logica di Business.

Questa documentazione ripercorre in modo strutturato tutte le fasi del progetto: dall'elicitazione dei requisiti e la modellazione visuale dei Casi d'Uso (UML), fino all'analisi delle scelte architetturali e alla documentazione delle strategie di validazione, ponendosi come sintesi pratica delle competenze teoriche acquisite durante il corso.

FASE DI IDEAZIONE E ANALISI DEI REQUISITI

Il progetto "Sistema di Noleggio Mezzi" nasce con l'obiettivo di informatizzare e centralizzare la gestione operativa di una compagnia di noleggio veicoli (auto, furgoni, veicoli elettrici). La fase di ideazione si è concentrata sulla necessità di creare un'architettura software in grado di supportare contemporaneamente due anime del business: le operazioni fisiche "al banco" (gestite dallo staff/cassiere) e le operazioni telematiche (prenotazioni via web da parte dei clienti).

Durante l'analisi dei requisiti, si è scelto di adottare un approccio di sviluppo iterativo e incrementale. Per garantire la solidità dell'architettura fondante, il primo ciclo di analisi si è concentrato esclusivamente sul "Core Business" del dominio, isolando e modellando i tre Casi d'Uso (UC) fondamentali per l'operatività di base:

1. **Visualizza Mezzi Disponibili:** Il primo requisito analizzato è stato la necessità di interrogare il sistema per conoscere lo stato dei mezzi inseriti. È emerso il bisogno di filtrare i veicoli non solo per tipologia o sede fisica di appartenenza, ma soprattutto in base al loro stato logico (escludendo veicoli già noleggiati, guasti o con batteria insufficiente). Questo caso d'uso ha gettato le basi per l'implementazione del *Pattern State* sui veicoli.
2. **Avvia Noleggio:** Si è analizzato il processo di assegnazione fisica di un veicolo a un cliente presente in filiale. I requisiti elicitati per questo scenario hanno imposto l'introduzione di controlli di sicurezza (verificare che il cliente sia "affidabile" e non sospeso) e di validazioni fisiche (es. impedire l'avvio del noleggio se il veicolo ha un livello di carica/carburante critico).
3. **Concludi Noleggio:** L'analisi della fase di rientro del veicolo ha fatto emergere la necessità di un motore di calcolo tariffario dinamico. Il sistema doveva essere in grado di calcolare il costo finale in base alla durata effettiva e ai chilometri

percorsi, scalare l'importo dal credito del cliente e, contestualmente, ripristinare la disponibilità del mezzo per i noleggi successivi. Ha inoltre introdotto i requisiti per la gestione delle anomalie (es. fallimento del pagamento e conseguente sospensione dell'account).

Evoluzione dell'Analisi: Una volta consolidati i requisiti e l'architettura per questi tre pilastri operativi, l'analisi è stata estesa ai requisiti secondari ma ugualmente critici, tra cui:

- La gestione delle prenotazioni future (che ha richiesto l'analisi delle sovrapposizioni temporali).
- Il ciclo di vita della manutenzione (UC8), per gestire i guasti e le perizie.
- La gestione della sicurezza e l'autenticazione degli attori per separare i privilegi di Clienti e Cassieri.

Questa scomposizione iniziale ha permesso di mitigare i rischi architetturali, assicurando che il motore di gestione dei noleggi fosse robusto prima di innestarvi logiche più complesse come l'incrocio delle date per le prenotazioni web.

MODELLO DEI CASI D'USO

Casi d'uso descritti in formato dettagliato:

UC1	AUTENTICA CASSIERE
Attore Primario	-Cassiere
Pre-Condizioni	Il sistema è attivo e raggiungibile. Il Cassiere è in possesso delle credenziali di accesso (ID/Username e Password).
Post-Condizioni	Il Cassiere è autenticato nel sistema. La sessione di lavoro è associata all'identificativo del Cassiere corrente. Viene mostrata la dashboard operativa.
Scenario Principale	1. Il Cassiere avvia l'applicazione o accede alla schermata di login. 2. Il Sistema mostra il modulo di autenticazione. 3. Il Cassiere inserisce le proprie credenziali (Username e Password). 4. Il Sistema valida la correttezza delle credenziali verificandole nel database.
Scenari Alternativi	4a. Credenziali non valide: • Il Sistema rileva che l'username non esiste o la password è errata.

	• Il Sistema mostra un messaggio di errore ("Credenziali errate"). • Il Sistema richiede di inserire nuovamente le credenziali (ritorno al passo 3). 4b. Account disabilitato/sospeso: • Il Sistema rileva che le credenziali sono corrette ma l'account del Cassiere è stato disabilitato (es. licenziamento o sospensione). • Il Sistema nega l'accesso e mostra un messaggio di contatto amministrativo.
--	---

UC2	REGISTRA CLIENTE
Attore Primario	-Impiegato
Pre-Condizioni	Il sistema è attivo e l'Impiegato è autenticato
Post-Condizioni	Un nuovo cliente è registrato nel sistema con uno stato di affidabilità iniziale.
Scenario Principale	1. L'impiegato seleziona l'opzione per registrare un nuovo cliente. 2. L'impiegato inserisce i dati anagrafici (Nome, Cognome, Documento, Metodo di Pagamento). 3. Il Sistema convalida i dati (formato corretto, unicità del documento). 4. Il Sistema crea il profilo cliente impostando lo stato di affidabilità su "Standard". 5. Il Sistema conferma l'avvenuta registrazione.
Scenari Alternativi	3a. Cliente già esistente: • Il Sistema rileva che il documento è già censito. • Il Sistema mostra i dati del cliente esistente. • L'impiegato può aggiornare i dati o annullare l'operazione. 3b. Dati di pagamento non validi: • Il Sistema segnala errore nella validazione della carta/metodo di pagamento. • L'impiegato richiede un altro metodo di pagamento.

UC3	VISUALIZZA DISPONIBILITA' MEZZI
Attore Primario	-Cassiere
Pre-Condizioni	Il sistema è attivo e raggiungibile. Il Cassiere è autenticato nel sistema. Il punto di noleggio corrente è identificato.
Post-Condizioni	Viene visualizzato l'elenco dei mezzi disponibili presso il punto di noleggio. Nessuna modifica permanente viene apportata ai dati.
Scenario Principale	1. Il Cassiere accede alla sezione 'Visualizza Disponibilità Mezzi'. 2. Il Sistema identifica il punto di noleggio corrente. 3. Il Sistema recupera l'elenco dei mezzi associati. 4. Il Sistema filtra i mezzi con stato 'Disponibile'. 5. Il Sistema mostra per ogni mezzo: ID, tipologia, livello carica/carburante, stato. 6. Il Cassiere consulta l'elenco. 7. Il Cassiere può selezionare un mezzo per avviare un noleggio.
Scenari Alternativi	4a. Nessun mezzo disponibile: • Il Sistema rileva assenza di mezzi disponibili. • Mostra un messaggio di avviso. • Suggerisce altri punti di noleggio (se previsto) 5a. Mezzo con carica insufficiente: • Il Sistema evidenzia il mezzo come non noleggiabile. • Mostra il livello sotto soglia minima.

UC4	Avvia Noleggio
Attore Primario	-Impiegato
Pre-Condizioni	Il cliente è registrato. Vi è almeno un mezzo disponibile nel punto noleggio corrente.
Post-Condizioni	Il noleggio è stato creato in stato 'Attivo', il mezzo è associato al noleggio e il suo stato è aggiornato a 'Noleggiato'
Scenario Principale	1. L'impiegato identifica il Cliente (tramite tessera o ricerca anagrafica). 2. Il Sistema mostra lo stato di affidabilità del Cliente ed eventuali pendenze.

	3. L'impiegato identifica il Mezzo da noleggiare (scansione codice o ID). 4. Il Sistema verifica che il Mezzo sia disponibile in loco e con carica/carburante sufficiente 5. L'impiegato seleziona la tipologia di Tariffa applicabile. 6. Il Sistema registra l'associazione Cliente-Mezzo, imposta lo stato del Mezzo su "Noleggiato" e l'ora di inizio. 7. Il Sistema genera la ricevuta di inizio noleggio.
Scenari Alternativi	2a. Cliente non affidabile / Pendenze aperte: • Il Sistema segnala che il Cliente ha penali non pagate o un rating troppo basso. • L'impiegato notifica il cliente e interrompe il noleggio (o richiede il saldo immediato). 2b. Cliente non registrato: • L'impiegato esegue il caso d'uso Registra Cliente • Si riprende dal passo 3. 4a. Mezzo non disponibile: • Il Sistema blocca il noleggio di quel specifico mezzo. • L'impiegato ne seleziona un altro. 4b. Mezzo Elettrico scarico: • Il Sistema rileva carica/carburante insufficiente. • Il Sistema aggiorna lo stato del mezzo in IN_RICARICA. • Il Sistema suggerisce un altro mezzo o impedisce il noleggio.

UC5	Concludi Noleggio
Attore Primario	-Impiegato
Pre-Condizioni	Esiste un noleggio attivo del mezzo in questione.
Scenario Principale	1. L'impiegato inserisce l'ID del Mezzo che viene riconsegnato.

	- Il Sistema recupera il Noleggio attivo e identifica il Punto Noleggio corrente (che potrebbe essere diverso da quello di partenza). - L'impiegato conferma lo stato fisico del mezzo (integro). - L'impiegato inserisce il livello di carica residua letto dal display. - Il Sistema calcola la durata del noleggio. - Il Sistema calcola il costo totale applicando la Strategia Tariffaria selezionata all'avvio. - Il Sistema verifica eventuali penali temporali (ritardo riconsegna). - Il Sistema aggiorna la posizione del Mezzo (nuovo Punto Noleggio) e lo stato in "Disponibile". - Il Sistema addebita l'importo al Cliente e chiude il Noleggio.
Scenari Alternativi	3a. Mezzo danneggiato: - L'impiegato segnala "Danno rilevato". - Il Sistema cambia lo stato del Mezzo in "In Manutenzione". - Il Sistema aggiunge una penale per danni al costo totale. 4a. Mezzo sottosoglia minima: - Il Sistema rileva carica/carburante critica. - Il Sistema imposta lo stato del Mezzo in "In Ricarica" (non disponibile per nuovi noleggi immediati). 7a. Ritardo eccessivo: - Il Sistema rileva che la durata supera il limite massimo consentito. - Il Sistema applica la penale per ritardo

UC6	SEGNALA GUASTO
Attore Primario	-Cassiere
Pre-Condizioni	Il Cassiere è autenticato nel sistema. Il Mezzo esiste nel sistema.
Post-Condizioni	Viene creata una nuova istanza di SegnalazioneGuasto collegata al Mezzo. Lo stato del Mezzo viene aggiornato a "In Manutenzione" (o

	"Non Disponibile"). Il mezzo viene rimosso dalla lista dei mezzi noleggiabili.
Scenario Principale	1. Il Cassiere avvia la procedura di segnalazione guasto. 2. Il Sistema richiede l'identificativo del Mezzo. 3. Il Cassiere inserisce l'ID del Mezzo (o scansiona il QR code). 4. Il Sistema recupera e mostra i dati del Mezzo (Modello, Stato attuale). 5. Il Cassiere inserisce i dettagli del problema (es. <i>Tipologia</i>: "Meccanico", <i>Descrizione</i>: "Catena rotta", <i>LivelloUrgenza</i>: "Alto"). 6. Il Sistema registra la segnalazione. 7. Il Sistema aggiorna lo stato del Mezzo in "In Manutenzione". 8. Il Sistema conferma l'operazione fornendo un ID ticket.
Scenari Alternativi	2a. Segnalazione contestuale al rientro (Estensione di UC5): • Questo scenario parte direttamente dal passo 5, poiché il mezzo è già stato identificato durante la procedura "Concludi Noleggio" 4a. Mezzo non trovato: • Il Sistema comunica che l'ID non corrisponde a nessun mezzo. • Il flusso termina o ritorna al passo 2. 7a. Mezzo già segnalato: • Il Sistema rileva che il mezzo è già nello stato "In Manutenzione". • Il Sistema chiede se aggiungere una nota alla segnalazione esistente invece di crearne una nuova.

UC7	AGGIUNGI NUOVO MEZZO
Attore Primario	-Cassiere
Pre-Condizioni	-Il Cassiere è stato autenticato
Post-Condizioni	Una nuova istanza di un veicolo specifico è stata salvata nel catalogo mezzi
Scenario Principale	1. Il Cassiere/Operatore accede alla sezione 'Aggiungi nuovo mezzo'. 2. Il Sistema richiede i dati identificativi (Targa/Seriale, Marca, Modello) e, fa selezionare la Categoria del Mezzo da un elenco predefinito (es. Auto, Furgone, Moto, Monopattino).Il Sistema recupera i dati del cliente e del noleggio. 3. L'Operatore inserisce i dati richiesti dal sistema. 4. Il sistema verifica che la Targa/Seriale non sia già presente nel Catalogo.

	5. Il Sistema crea il nuovo veicolo con le caratteristiche precedentemente specificate 6. Il Sistema imposta lo stato del veicolo su 'Disponibile'. 7. Il Sistema aggiunge il veicolo al Catalogo Mezzi 8. Il Sistema comunica il successo dell'operazione
Scenari Alternativi	4a. Targa già esistente: • Il Sistema rileva che la targa inserita appartiene già a un mezzo a catalogo. Il Sistema avvisa il Gestore dell'errore e interrompe la creazione, riportandolo al passo 2. 4b. Dati incompleti o malformati: • Il Sistema rileva che alcuni campi obbligatori sono vuoti o la targa non rispetta il formato. Mostra un messaggio di errore e richiede la correzione. 5a. Categoria mezzo non riconosciuta: • Se per qualche motivo viene passata una categoria non valida, il Sistema blocca la creazione, lancia un'eccezione interna e avvisa il Gestore con un messaggio generico di errore.

UC8	EFFETTUA MANUTENZIONE
Attore Primario	-Operatore Manutenzione
Pre-Condizioni	Il sistema è attivo e l'Operatore è autenticato. Esiste almeno un Mezzo nel sistema in stato In Manutenzione con una Segnalazione associata oppure una manutenzione programmata.
Post-Condizioni	L'intervento di manutenzione è registrato. Lo stato del mezzo viene aggiornato (Disponibile / Fuori Servizio).
Scenario Principale	1. L'Operatore accede alla sezione 'Gestione Manutenzione'. 2. Il Sistema mostra l'elenco dei mezzi In Manutenzione. 3. L'Operatore seleziona il mezzo da mantenere. 4. Il Sistema mostra i dati del mezzo, lo storico manutenzioni e l'ultima segnalazione aperta. 5. L'Operatore esegue la riparazione e inserisce a sistema la tipologia di intervento e la descrizione delle attività svolte (es. "Sostituzione specchietto"). 6. L'Operatore indica data e costo dell'intervento. 7. L'Operatore richiede l'aggiornamento dello stato del mezzo in Disponibile. 8. Il Sistema registra la manutenzione, archivia la Segnalazione, aggiorna lo stato del mezzo e conferma l'operazione.
Scenari Alternativi	2a. Nessun mezzo in manutenzione:

	- Il Sistema segnala che non ci sono mezzi che necessitano di interventi (lista vuota). 8a. Mezzo non idoneo (danno irreparabile): - Al passo 7, l'Operatore constata che il mezzo non è riparabile e richiede l'aggiornamento in Fuori Servizio. - Il Sistema registra l'intervento a costo zero, imposta lo stato a FUORI_SERVIZIO (o lo rimuove dal catalogo) e conferma l'operazione.
--	---

UC9	EFFETTUA PRENOTAZIONE WEB
Attore Primario	-Cliente registrato
Pre-Condizioni	Il Cliente ha effettuato il Login (è autenticato).
Post-Condizioni	Prenotazione salvata e associata all'utente corrente.
Scenario Principale	- Il Cliente accede alla sezione "Prenota Ora" della Dashboard. - Il Cliente inserisce i criteri di ricerca: Date (Inizio/Fine) e Luogo di Ritiro. - Il Cliente preme "Cerca". - Il Sistema verifica la disponibilità dei mezzi per quel periodo. - Il Sistema mostra l'elenco delle Categorie Disponibili con i relativi preventivi. - Il Cliente seleziona la Categoria desiderata. - Il Sistema mostra il riepilogo e chiede conferma. - Il Cliente conferma la prenotazione. - Il Sistema recupera l'identità del cliente dalla Sessione di Sicurezza (senza chiederla all'utente) e salva la prenotazione. - Il Sistema mostra il messaggio "Prenotazione Confermata" con il codice PNR.
Scenari Alternativi	4a. Nessuna disponibilità: - Il Sistema mostra un avviso: "Nessun veicolo disponibile per le date selezionate". - Il Sistema suggerisce le date libere più vicine. 8a. Sessione Scaduta (Timeout): - Il Cliente preme conferma, ma il token è scaduto. - Il Sistema reindirizza alla pagina di Login.

	Dopo il login, il Sistema ripristina la prenotazione in sospeso (User Experience avanzata).
--	---

UC10	GESTISCI BLACKLIST
Attore Primario	- Amministratore
Pre-Condizioni	Il cliente è registrato nel sistema. L'Amministratore è autenticato con privilegi di gestione.
Post-Condizioni	Il cliente viene inserito o rimosso dalla blacklist. Eventuali tentativi futuri di noleggio risultano bloccati se il cliente è in blacklist.
Scenario Principale	1. L'Amministratore accede alla sezione 'Gestisci Blacklist'. 2. Ricerca il cliente tramite ID, documento o anagrafica. 3. Il Sistema mostra i dati del cliente e lo storico (penali, incidenti, morosità). 4. L'Amministratore seleziona l'opzione di inserimento in blacklist. 5. Inserisce la motivazione del provvedimento. 6. Il Sistema registra l'operazione. 7. Il Sistema aggiorna lo stato del cliente. 8. Il Sistema blocca la possibilità di avviare nuovi noleggi.
Scenari Alternativi	4a. Cliente già in blacklist: • Il Sistema segnala che il cliente risulta già bloccato. 4b. Rimozione dalla blacklist: • L'Amministratore seleziona la riabilitazione. • Inserisce la motivazione. • Il Sistema aggiorna lo stato consentendo nuovi noleggi.

GLOSSARIO

- **Mezzo**

Bene fisico noleggiabile (es. auto, SUV, furgone, monopattino) registrato nel sistema. È identificato da un ID univoco e possiede caratteristiche tecniche, una sede di appartenenza e uno stato operativo in tempo reale.

- **Punto Noleggio**

Sede fisica (filiale) dell'azienda in cui il cliente può ritirare o riconsegnare un mezzo.

- **Cliente**

Utente registrato nel sistema gestionale, abilitato a effettuare ricerche, prenotazioni e noleggi. È caratterizzato da un livello di affidabilità che ne determina i permessi.

- **Operatore / Cassiere**

Membro dello staff autorizzato ad accedere al sistema di back-office per gestire registrazioni, avviare o concludere noleggi, e aggiornare il catalogo mezzi.

- **Prenotazione**

Impegno formale che blocca un mezzo per un periodo di tempo futuro (Data Inizio e Data Fine). Genera un preventivo di costo ma non avvia l'effettivo conteggio dei chilometri.

- **PNR (Codice di Prenotazione)**

Codice alfanumerico univoco (es. di 6 caratteri) generato automaticamente dal sistema per identificare e recuperare in modo univoco una prenotazione confermata.

- **Noleggio**

Contratto attivo che traccia l'effettivo utilizzo del mezzo. Inizia con il ritiro fisico (registrando data, ora e chilometri o carica iniziale) e si conclude con la riconsegna (calcolando i consumi/percorrenze e il tempo reale di utilizzo).

- **Tariffa**

Insieme di regole e algoritmi (es. tariffa oraria, giornaliera, chilometrica) applicati dal sistema per calcolare dinamicamente il costo stimato (preventivo) o il costo effettivo finale di un noleggio.

- **Catalogo Mezzi**

Registro centralizzato del sistema che contiene l'elenco di tutta la flotta aziendale. Filtra e traccia in tempo reale quali veicoli sono liberi, occupati o guasti.

- **Stato Mezzo**

Condizione operativa attuale di un veicolo, gestita in modo rigoroso dal sistema. Può assumere valori come: *Disponibile*, *Noleggiato*, *In Manutenzione* o *Fuori Servizio*.

- **Livello di Carica**
Percentuale di batteria residua associata ai mezzi elettrici (o livello di carburante per mezzi termici). Costituisce un parametro vincolante per la disponibilità del mezzo (es. non noleggiabile se inferiore al 20%).
- **Segnalazione (Guasto)**
Report testuale creato da un cliente o da un operatore per tracciare un malfunzionamento o un danno estetico su un veicolo. Causa il blocco preventivo del mezzo per ragioni di sicurezza.
- **Intervento di Manutenzione**
Registrazione formale di un'operazione di riparazione effettuata in officina. Entra a far parte dello storico del veicolo, tracciandone la descrizione e il costo di ripristino.
- **Blacklist (Sospensione Account)**
Stato disciplinare in cui un cliente viene temporaneamente inibito dal sistema (es. per insolvenza, ritardi gravi o danni causati). Impedisce all'utente di effettuare nuove prenotazioni.
- **Tipo / Descrizione Mezzo**
Insieme delle caratteristiche tecniche (marca, modello, anno di immatricolazione, numero di posti, categoria veicolo) che categorizzano un mezzo e permettono ai clienti di applicare filtri durante la ricerca.

LOGICA DI BUSINESS E REGOLE DI DOMINIO

La logica di business del sistema "Noleggio Mezzi" è stata centralizzata all'interno del **Service Layer** (es. PrenotazioneService, NoleggioService, MezzoService). Questa scelta architetturale garantisce un forte disaccoppiamento tra l'interfaccia utente (Controller) e l'accesso ai dati (Repository), mantenendo il codice modulare e facilmente testabile.

Di seguito sono descritte le principali regole di dominio e i Design Pattern implementati per farle rispettare:

- **Ciclo di Vita del Veicolo (State Pattern)**
Il controllo sullo stato fisico e logico dei veicoli è gestito tramite il **Pattern State**. Un veicolo non è una semplice entità passiva, ma delega i propri comportamenti alla sua classe di stato corrente (es. StatoDisponibile, StatoNoleggiato, StatoInManutenzione).
- **Regola di Dominio:** Il sistema impedisce a livello architetturale operazioni fisicamente impossibili. Ad esempio, tentare di noleggiare o prenotare un veicolo in StatoInManutenzione solleverà automaticamente una StatoNonValidoException, senza che il Controller debba effettuare controlli espliciti.

- **Motore di Ricerca e Prenotazione (Sovrapposizione Temporale)**

L'algoritmo di disponibilità dei veicoli si basa sull'incrocio dinamico tra il parco auto fisicamente presente in una Sede e le prenotazioni già registrate.

- **Regola di Dominio:** Un veicolo viene mostrato nei risultati di ricerca solo se il periodo richiesto dall'utente non presenta alcuna sovrapposizione (`DateRange.sovrappone()`) con i periodi in cui il veicolo è già impegnato in altre prenotazioni attive.

- **Affidabilità e Sicurezza del Cliente (RBAC e Sospensioni)**

Il sistema integra un controllo degli accessi basato sui ruoli (Role-Based Access Control) implementato tramite un `AuthInterceptor`, che divide nettamente le rotte web pubbliche, quelle dedicate ai Clienti e quelle ad uso esclusivo dello Staff (Cassieri/Amministratori).

- **Regole di Dominio:** Un cliente viene considerato abilitato ai servizi solo se il suo flag `isAffidabile()` risulta vero. In caso di mancato saldo al termine di un noleggio (credito insufficiente) o in caso di segnalazione furto/sinistro, il sistema solleva un'eccezione, incamera il veicolo e **sospende automaticamente e immediatamente** l'account del cliente per motivi di sicurezza.

- **Calcolo Dinamico delle Tariffe (Decorator Pattern)**

Per evitare una proliferazione di classi dovuta alle innumerevoli combinazioni tariffarie, il calcolo dei costi è stato modellato utilizzando il **Pattern Decorator**.

- **Regola di Dominio:** Il costo finale di un noleggio viene calcolato partendo da una tariffa base (es. `TariffaOraria`, `TariffaGiornaliera`), a cui possono essere "agganciati" dinamicamente dei decorator a runtime, come la `PenaleGiovaneGuidatore` o l'`AssicurazioneFurto`, che ricalcolano l'importo totale in modo trasparente per il sistema.

- **Vincoli di Creazione ed Esercizio (Builder Pattern)**

La creazione di nuovi mezzi per l'ampliamento della flotta richiede parametri complessi (dati di immatricolazione, tipo, sede fisica di allocazione).

- **Regola di Dominio:** La corretta istanziatura è demandata al `MezzoBuilder`, che applica validazioni formali (es. obbligo di marca/modello, controlli sull'anno di immatricolazione) e impedisce l'inserimento nel `CatalogoMezzi` di veicoli con dati corrotti o incompleti. Inoltre, in fase operativa, il sistema impedisce il noleggio di veicoli elettrici o monopattini che possiedono un livello di batteria inferiore a una soglia limite di sicurezza (es. 20%).

ANALISI ORIENTATA AGLI OGGETTI

Seguendo l'approccio iterativo evolutivo, la realizzazione dell'applicazione è stata articolata in diverse iterazioni. Per ciascuna iterazione ci si è occupati di gestire specifici casi d'uso, partendo dall'analisi delle problematiche fino alla progettazione delle classi e delle sequenze.

ITERAZIONE 1

UC3: Visualizza Disponibilità Mezzi

Problematiche:

- Il cliente ha la necessità di interrogare il sistema per scoprire quali veicoli sono liberi in un determinato periodo (Data Inizio / Data Fine) e presso una specifica sede di ritiro.
- Il sistema deve filtrare dinamicamente i mezzi, escludendo quelli già noleggiati, in manutenzione o fuori servizio.
- Insieme alla lista dei mezzi, il sistema deve fornire un **preventivo di costo**. Tuttavia, le logiche di tariffazione (oraria, giornaliera, chilometrica) possono variare, e cablare (hard-code) queste formule matematiche direttamente nel codice renderebbe il sistema rigido e difficile da aggiornare in futuro.

Soluzioni:

- Implementazione di un modulo di ricerca all'interno del controller, che interroga il CatalogoMezzi per filtrare i veicoli in base allo stato (StatoMezzo.DISPONIBILE) e alla posizione.
- **Applicazione del Pattern Strategy:** Per la generazione del preventivo, è stata introdotta un'interfaccia ITariffa (la "Strategia"). Le classi concrete (es. TariffaGiornaliera, TariffaOraria) implementano questa interfaccia. Il sistema delega il calcolo del costo alla specifica tariffa associata, favorendo il principio Open/Closed (aperto all'estensione, chiuso alla modifica) e permettendo di calcolare preventivi personalizzati senza alterare la logica di ricerca.

MODELLO DELLE CLASSI

Relativamente al caso d'uso di visualizzazione della disponibilità, la progettazione ha portato all'individuazione delle seguenti classi e pattern:

- **PrenotazioneController:** Controller (<<control>>) che espone il metodo cercaDisponibilita(). Riceve i parametri di ricerca dall'interfaccia utente e coordina la logica di dominio.
- **MezzoService:** Classe di servizio che incapsula la logica di business core relativa ai mezzi. Riceve la richiesta dal controller, interroga il CatalogoMezzi per l'effettivo filtraggio e coordina il calcolo del preventivo, alleggerendo il controller dalle operazioni complesse.

- **CatalogoMezzi:** Repository incaricato di filtrare l'effettiva disponibilità fisica dei mezzi (controllando sede, tipo e StatoMezzo).
- **DateRange:** Classe Value Object utilizzata per incapsulare in modo sicuro le date di inizio e fine ricerca, facilitando il calcolo della durata del noleggio.
- **ITariffa (Interfaccia) e Classi Concrete:** Architettura basata sul *Design Pattern Strategy*. L'interfaccia definisce il contratto `calcolaCosto(durata, km)`, mentre le classi come `TariffaGiornaliera` applicano l'algoritmo specifico restituendo il preventivo al cliente.

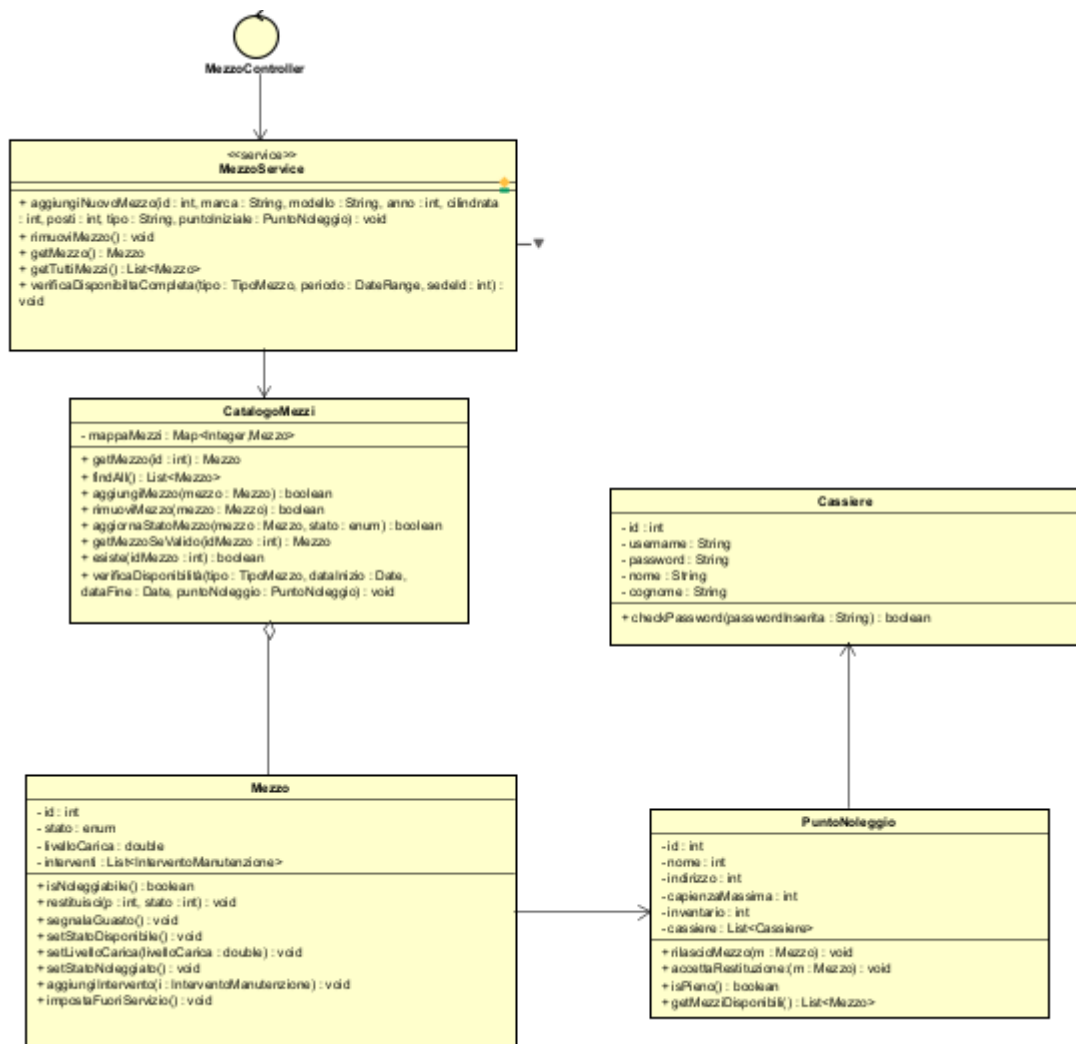
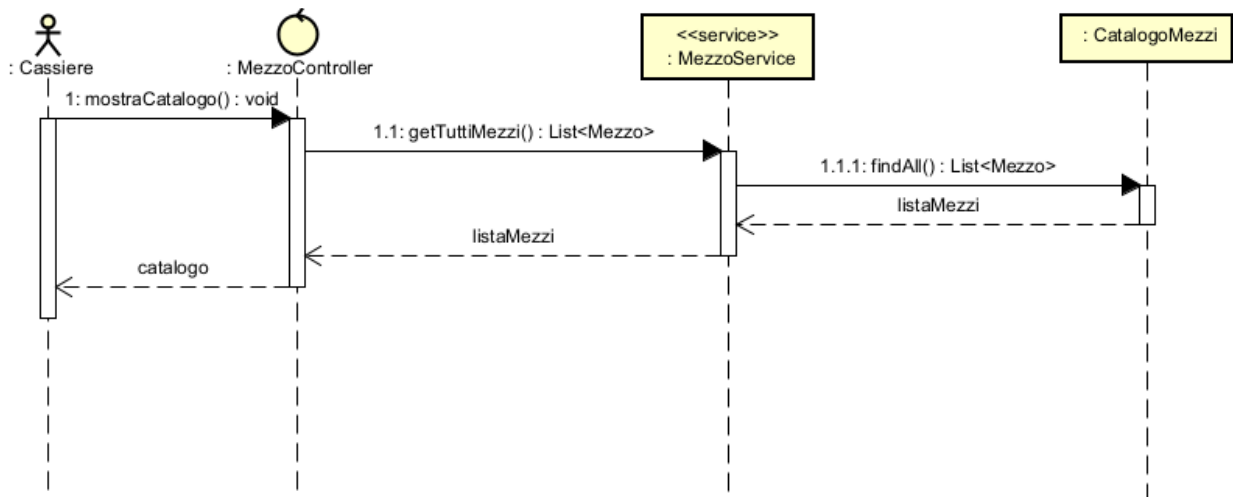


DIAGRAMMA DI SEQUENZA E CONTRATTI DELLE OPERAZIONI

Il diagramma di sequenza mostra l'interazione tra l'Attore e il Sistema durante la fase di interrogazione del catalogo, culminando con la restituzione della lista dei mezzi liberi (se presenti) o con un'eccezione dedicata in caso di esaurimento veicoli.



Di seguito viene definito il contratto per l'operazione di sistema principale innescata dall'attore per la ricerca.

CONTRATTO CO1: cercaDisponibilita

- **OPERAZIONE:** cercaDisponibilita (tipo: TipoMezzo, periodo: DateRange, idPuntoNoleggio: int)
- **RIFERIMENTI:** Visualizza Disponibilità Mezzi
- **PRECONDIZIONI:** * Il sistema è in funzione e il catalogo dei mezzi è caricato in memoria.
 - Il Cliente ha inserito un periodo temporale valido (Data Fine > Data Inizio).
- **POST CONDIZIONI:**
 - Il sistema invoca il metodo verificaDisponibilita sul CatalogoMezzi.
 - **Se vengono trovati veicoli compatibili:** Il sistema calcola il preventivo per ciascun mezzo tramite il pattern Strategy (ITariffa) applicato alla durata estratta da DateRange. L'operazione restituisce una List<Mezzo> contenente i veicoli liberi da mostrare a video.
 - **Se non ci sono veicoli disponibili (Scenario Alternativo):** L'operazione solleva una NessunaDisponibilitaException e il sistema notifica all'utente l'assenza di mezzi per i criteri selezionati.

UC4: Avvia Noleggio

Problematiche:

- L'azienda deve registrare l'effettivo momento in cui il cliente ritira il veicolo, facendo partire formalmente il noleggio.
- È fondamentale tracciare i dati iniziali del mezzo (data e ora esatta di ritiro, chilometraggio di partenza o livello di carica della batteria) per poter calcolare correttamente la tariffa finale al momento della restituzione.
- Il sistema deve aggiornare in tempo reale lo stato del veicolo per impedire che venga assegnato per errore ad altri clienti.
- Necessità di verificare che il cliente sia regolarmente censito a sistema e che non abbia pendenze o sospensioni (account non affidabile) prima di consegnargli le chiavi.

Soluzioni:

- Implementazione di un modulo di gestione noleggi gestito dal `NoleggioController`, che fa da intermediario tra l'operatore e il database in memoria.
- Creazione dell'entità `Noleggio` per storicizzare in modo immutabile le condizioni di partenza.
- Coordinamento tra il `RegistroNoleggi` (per il salvataggio della transazione) e il `CatalogoMezzi` (per il cambio di stato del veicolo in `NOLEGGIATO`).

MODELLO DELLE CLASSI

Relativamente al caso d'uso di avvio del noleggio, la progettazione ha portato all'individuazione delle seguenti classi concettuali:

- **NoleggioController:** Rappresenta il controller d'interfaccia (`<<control>>`). Gestisce la richiesta di inizio noleggio, validando gli input dell'operatore e richiamando i servizi di dominio.
- **NoleggioService:** Classe di servizio che incapsula la logica di business. È responsabile di coordinare le verifiche (es. controllare se il cliente è affidabile e se il mezzo è disponibile), istanziare la nuova entità `Noleggio` con i dati di partenza e richiamare i repository per il salvataggio.
- **RegistroNoleggi e CatalogoMezzi:** Repositories incaricati, rispettivamente, di salvare il contratto di noleggio in corso e di recuperare/aggiornare l'istanza del veicolo interessato.
- **Noleggio:** Entità di dominio che modella il contratto attivo. Contiene i riferimenti al Cliente, al Mezzo, la data/ora di inizio (`LocalDateTime.now()`) e i chilometri di partenza.
- **Cliente e Mezzo:** Entità coinvolte nel processo. Il sistema verificherà che il Cliente sia idoneo (es. `isAffidabile()`) e che il Mezzo sia idoneo al ritiro.

- Il Cliente esiste nel registro ed è in uno stato valido (non sospeso).
- Il Mezzo si trova nel CatalogoMezzi nello stato DISPONIBILE (o PRENOTATO da quello stesso cliente) e ha un livello di batteria/carburante sufficiente.
- **POST CONDIZIONI:**
 - Viene creata una nuova istanza dell'entità Noleggio associata al cliente e al mezzo (freccia <<create>>).
 - I dati iniziali (data odierna e kmIniziali) vengono registrati nell'oggetto Noleggio.
 - Il sistema invoca aggiungiNoleggio(n) sul RegistroNoleggi per memorizzare la transazione attiva.
 - Il sistema aggiorna lo stato del Mezzo all'interno del CatalogoMezzi impostandolo su NOLEGGIATO.
 - L'operazione restituisce una conferma a video (es. riepilogo del contratto avviato) all'operatore.

UC5: Concludi Noleggio

Problematiche:

- Al momento della riconsegna del veicolo, il sistema deve chiudere formalmente il contratto di noleggio precedentemente aperto.
- È necessario calcolare in modo esatto la tariffa finale da far pagare al cliente. Questo calcolo non è più una stima (come nel preventivo della prenotazione), ma deve basarsi sulla durata effettiva e sui chilometri realmente percorsi (kmFinali - kmIniziali).
- Il sistema deve gestire lo stato in cui viene restituito il veicolo: se è integro, deve tornare immediatamente noleggiabile; se presenta danni, deve essere bloccato per evitare di consegnarlo al cliente successivo.
- Occorre eventualmente addebitare il costo sul conto del cliente o registrarne il pagamento, valutando modifiche al suo livello di affidabilità in caso di danni o forti ritardi.

Soluzioni:

- Implementazione di una logica di chiusura all'interno del NoleggioController, che recupera il contratto attivo dal RegistroNoleggi.
- **Riuso del Pattern Strategy:** Il calcolo finale viene delegato nuovamente all'interfaccia ITariffa. Passando alla tariffa i minuti reali trascorsi e i chilometri effettivi calcolati, il sistema restituisce l'importo preciso, garantendo coerenza con il modello architetturale delineato nell'UC9.
- Aggiornamento condizionale dello stato nel CatalogoMezzi: il veicolo passa a DISPONIBILE oppure a IN_MANUTENZIONE (in caso di segnalazione danni da parte dell'operatore).

MODELLO DELLE CLASSI

Relativamente al caso d'uso di chiusura del noleggio, la progettazione ha coinvolto le seguenti classi concettuali:

- **NoleggioController:** Controller (<<control>>) che gestisce la transazione di chiusura. Riceve i dati di riconsegna dall'interfaccia utente (km finali, presenza di danni) e orchestra il processo.
- **NoleggioService:** Classe di servizio che incapsula la logica di business. È responsabile di coordinare le verifiche (es. controllare se il cliente è affidabile e se il mezzo è disponibile), istanziare la nuova entità Noleggio con i dati di partenza e richiamare i repository per il salvataggio.
- **RegistroNoleggi:** Fornisce il metodo per recuperare l'istanza specifica del Noleggio tramite il suo identificativo univoco.
- **Noleggio:** Entità che viene aggiornata. Registra la dataFine, calcola i kmPercorsi e invoca il metodo calcolaCosto() della strategia tariffaria associata (ITariffa).
- **CatalogoMezzi:** Riceve la direttiva per aggiornare lo stato del mezzo (aggiornaStatoMezzo).
- **Cliente:** Riceve l'addebito finale tramite il metodo addebitaImporto(importo).

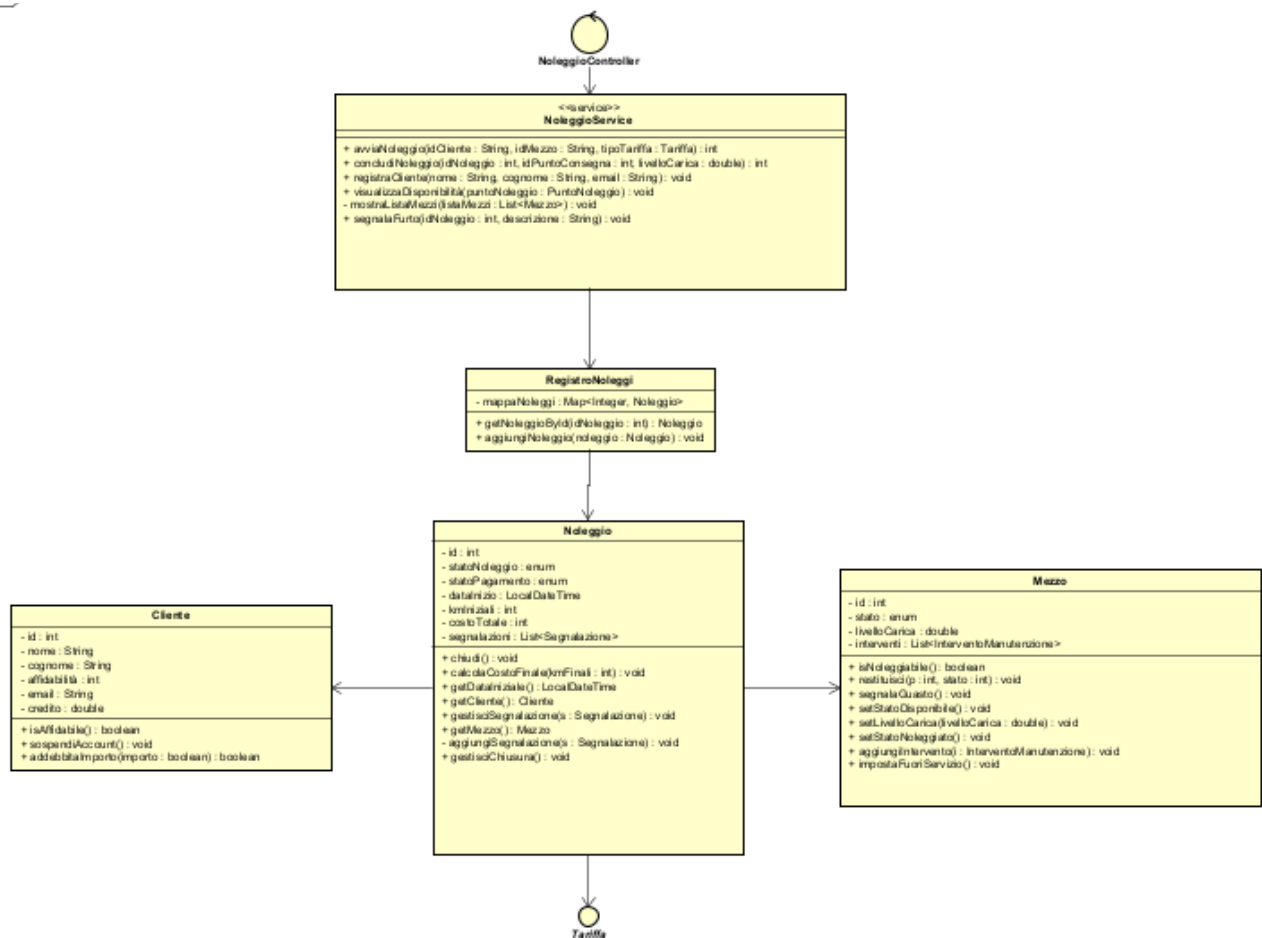
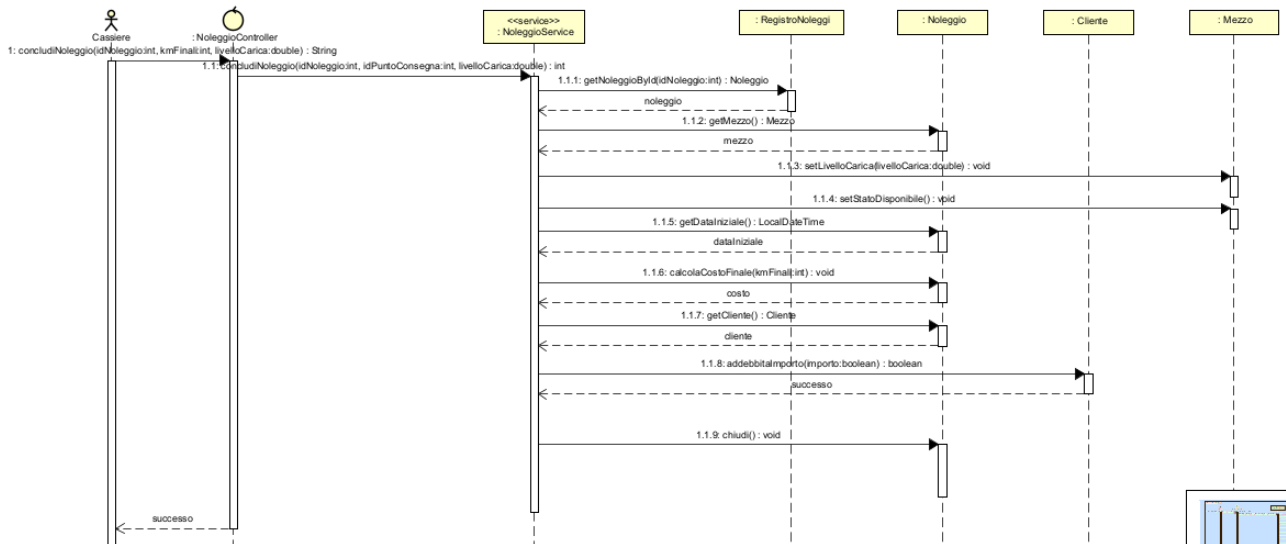


DIAGRAMMA DI SEQUENZA E CONTRATTI DELLE OPERAZIONI

Il diagramma di sequenza illustra l'interazione per la chiusura del contratto. Evidenzia il recupero dell'entità, l'invocazione del calcolo matematico della tariffa e la sequenza di aggiornamenti di stato su mezzo e cliente.



Di seguito viene specificato formalmente il contratto per l'operazione di sistema principale.

CONTRATTO CO1: concludiNoleggio

- **OPERAZIONE:** concludiNoleggio (idNoleggio: int, kmFinali: int, danniRilevati: boolean)
- **RIFERIMENTI:** Concludi Noleggio
- **PRECONDIZIONI:** * Il sistema è in funzione.
 - L'Operatore (Cassiere) è autenticato.
 - Il contratto identificato da idNoleggio esiste nel RegistroNoleggi e si trova in stato "Attivo" (non ancora concluso).
 - Il parametro kmFinali è maggiore o uguale ai chilometri iniziali registrati sul contratto.
- **POST CONDIZIONI:**
 - L'istanza di Noleggio registra la data e l'ora attuali come termine del contratto.
 - Il costo totale viene calcolato in via definitiva tramite il pattern Strategy (ITariffa) moltiplicando/applicando la durata effettiva e i kmFinali - kmIniziali.
 - Il Noleggio viene contrassegnato come "Concluso".
 - Il sistema invoca addebitaImporto(costoTotale) sull'oggetto Cliente associato.

- **Se danniRilevati è falso:** Il sistema aggiorna lo stato del Mezzo nel CatalogoMezzi a DISPONIBILE.
- **Se danniRilevati è vero:** Il sistema aggiorna lo stato del Mezzo a IN_MANUTENZIONE (attivando la pre-condizione per l'UC8 Manutenzione).
- L'operazione restituisce il riepilogo della ricevuta/fattura generata.

ITERAZIONE 2

UC1: Autentica Cassiere

Problematiche:

- Il sistema gestionale deve garantire l'accesso esclusivo al personale autorizzato (Cassieri/Operatori), prevenendo accessi indesiderati ai dati sensibili e alle operazioni di business.
- Necessità di gestire scenari alternativi come l'inserimento di credenziali errate o il tentativo di accesso da parte di un account disabilitato o sospeso.

Soluzioni:

- Implementazione di un modulo di Login dedicato che riceve le credenziali e le valida attraverso un servizio di autenticazione.
- Utilizzo del pattern MVC, delegando all'entità Cassiere la responsabilità di verificare la corrispondenza della propria password.

MODELLO DELLE CLASSI

Relativamente al caso d'uso UC1, dopo un'attenta analisi e progettazione, è stato possibile individuare le seguenti classi e interfacce:

- **LoginController:** Rappresenta il controller d'interfaccia (<<control>>). Gestisce le richieste di accesso provenienti dalla schermata di login e le delega al livello di servizio.
- **AutenticazioneService:** Classe che racchiude la logica di business relativa all'autenticazione. Coordina la ricerca dell'utente e la verifica delle credenziali.
- **CassiereRepository:** Interfaccia (<<interface>>) responsabile della persistenza e del recupero dei dati. Contiene il metodo per cercare un utente specifico tramite il suo username.
- **Cassiere:** Rappresenta l'entità principale (Utente/Staff). Contiene le informazioni personali (id, nome, cognome, username, password) ed espone il metodo checkPassword() per incapsulare la logica di validazione della sicurezza.

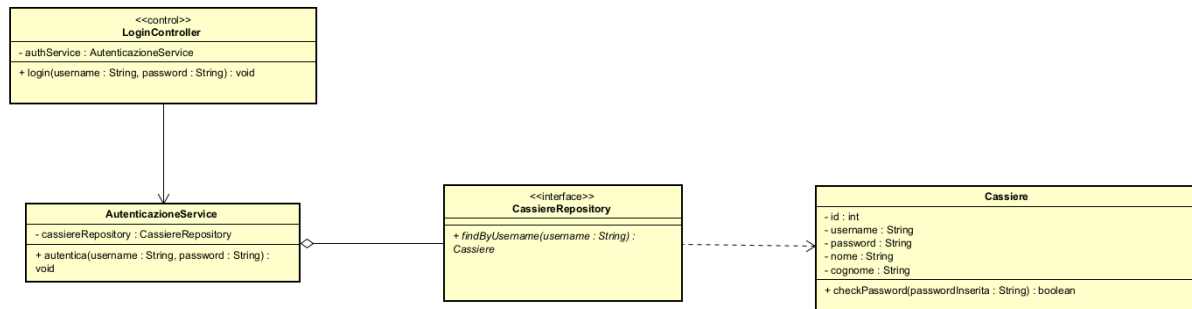
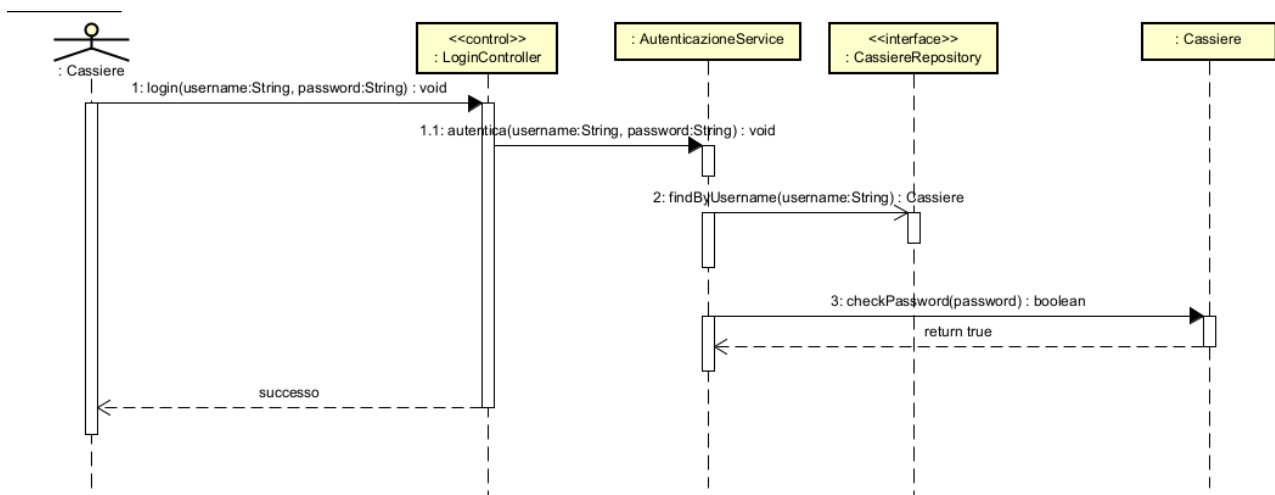


DIAGRAMMA DI SEQUENZA E CONTRATTI DELLE OPERAZIONI

Il diagramma di sequenza descrive visivamente il flusso delle operazioni e dei messaggi scambiati tra le entità del sistema durante il processo di login.



Di seguito viene specificato formalmente il contratto per l'operazione di sistema principale innescata dall'attore.

CONTRATTO CO1: login

- **OPERAZIONE:** login (username: String, password: String)
- **RIFERIMENTI:** Autentica Cassiere (UC1)
- **PRECONDIZIONI:** * Il sistema è attivo e raggiungibile.
 - Il Cassiere non è ancora autenticato e si trova nella schermata di login.
- **POST CONDIZIONI:**
 - Il sistema invoca findByUsername per recuperare i dati dell'account dal database.

- Il sistema esegue checkPassword passando la password inserita.
- **Se le credenziali sono corrette e l'account è attivo:** l'operazione restituisce true (successo), la sessione viene avviata e l'utente ha accesso alla dashboard.
- **Se le credenziali sono errate:** l'operazione fallisce, l'accesso viene negato e il sistema notifica all'utente l'errore ("Credenziali errate").
- **Se l'account è disabilitato:** l'accesso viene negato e viene mostrato un messaggio di contatto amministrativo.

UC2: Registrazione Cliente

Problematiche:

- L'azienda necessita di censire i nuovi clienti all'interno del sistema gestionale per abilitarli alle future operazioni di noleggio mezzi.
- È fondamentale evitare la duplicazione dei profili (stesso cliente registrato più volte) e garantire che ogni cliente abbia uno "stato di affidabilità" tracciato fin dal primo inserimento.
- Necessità di un sistema centralizzato per recuperare, aggiungere o sospendere gli account dei clienti.

Soluzioni:

- Implementazione di un modulo di registrazione gestito da un controller dedicato, che fa da tramite tra l'interfaccia dell'impiegato e la logica di dominio.
- Creazione di un RegistroClienti che funge da repository in memoria, incaricato di mantenere l'elenco dei clienti, validarne l'unicità e gestirne il ciclo di vita.

MODELLO DELLE CLASSI

Relativamente al caso d'uso UC2, la progettazione ha portato all'individuazione delle seguenti classi concettuali:

- **ClienteController:** Rappresenta il controller (<<control>>) deputato a gestire la richiesta di registrazione. Intercetta i dati anagrafici inseriti dal cassiere e coordina la creazione dell'oggetto e il suo salvataggio.
- **ClienteService:** Classe di servizio che incapsula la logica di business. Riceve i dati dal controller, coordina la creazione del nuovo oggetto Cliente (assegnando eventuali valori di default come il livello di affidabilità iniziale) e interagisce con il registro per il salvataggio, gestendo eventuali controlli sui duplicati.
- **RegistroClienti:** Classe che agisce come gestore della collezione dei clienti (tramite una mappa mappaClienti). Offre i metodi necessari per l'inserimento (aggiungiCliente), la verifica dell'esistenza (isRegistrato) e la gestione dello stato (rimuoviCliente, getAffidabilitàCliente).

- **Cliente:** Rappresenta l'entità principale di dominio. Contiene sia i dati anagrafici (id, nome, cognome, email), sia le variabili di stato e di business (affidabilità, credito). Espone metodi critici come `isAffidabile()`, `sospendiAccount()` e `addebitaImporto()`.

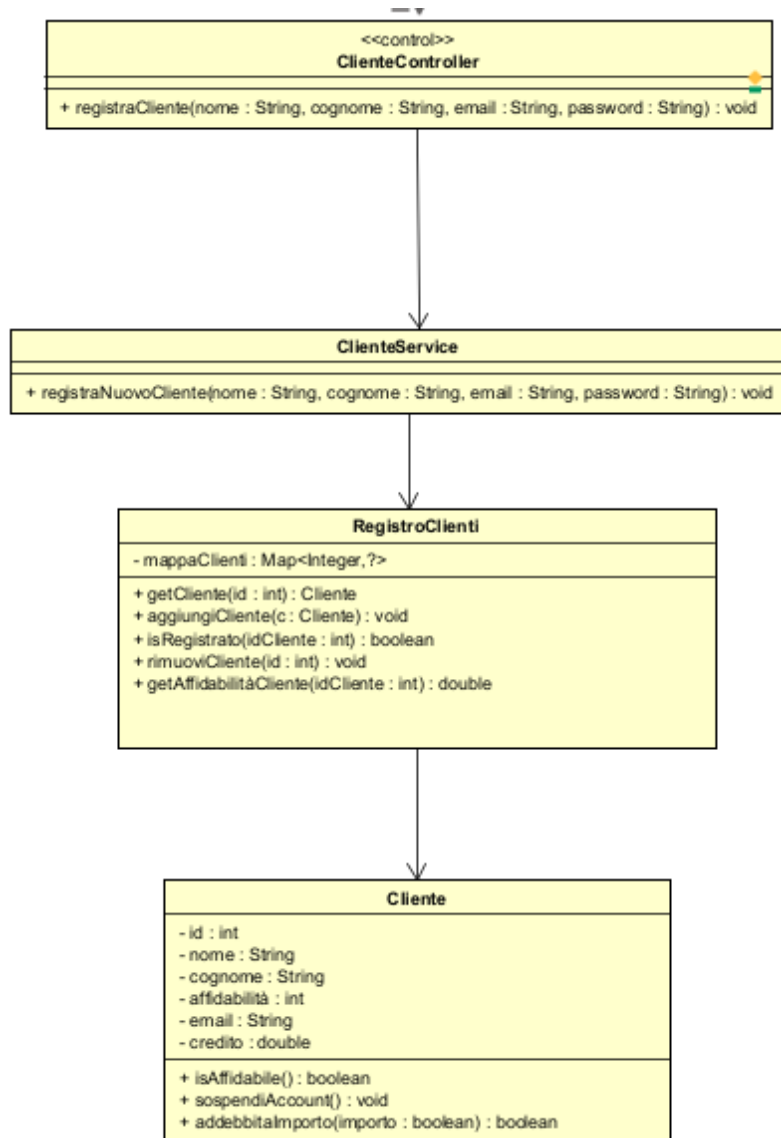
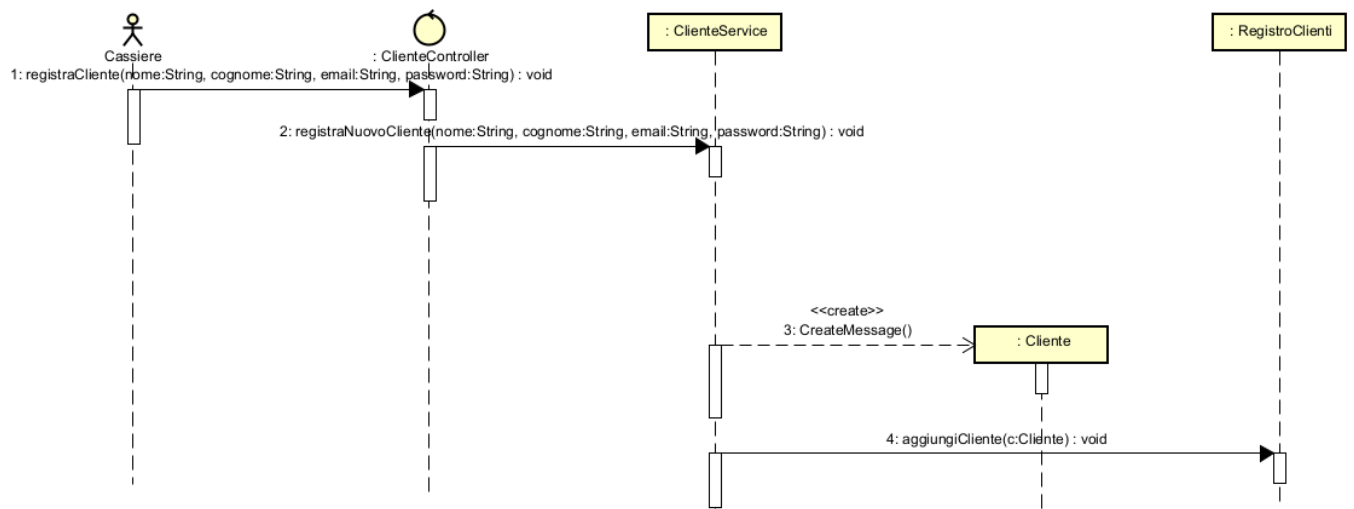


DIAGRAMMA DI SEQUENZA E CONTRATTI DELLE OPERAZIONI

Il diagramma di sequenza illustra la dinamica di creazione di un nuovo profilo cliente. Il flusso mostra l'interazione diretta tra il Cassiere, il Controller e il Registro per il salvataggio permanente.



Di seguito viene definito il contratto per l'operazione di sistema principale.

CONTRATTO CO1: **registraCliente**

- **OPERAZIONE:** registraCliente (nome: String, cognome: String, email: String)
- **RIFERIMENTI:** Registra Cliente (UC2)
- **PRECONDIZIONI:** * Il sistema è attivo.
 - Il Cassiere (Impiegato) ha effettuato l'accesso con successo ed è autenticato.
 - I dati inseriti nel modulo hanno superato un primo controllo di formato lato interfaccia.
- **POST CONDIZIONI:**
 - Il ClienteController istanzia un nuovo oggetto Cliente tramite il costruttore (freccia <<create>>).
 - Il sistema assegna automaticamente al nuovo cliente un livello di affidabilità iniziale (es. "Standard") e un ID univoco.
 - Il controller invoca il metodo aggiungiCliente(c: Cliente) sul RegistroClienti per memorizzare il profilo nella mappaClienti.
 - Viene restituita conferma a video della corretta creazione del profilo.

UC7: Aggiungi Nuovo Mezzo

Problematiche:

- La flotta aziendale è in continua evoluzione e il sistema deve permettere l'inserimento di nuovi veicoli all'interno del database gestionale.
- È necessario garantire che ogni nuovo veicolo sia associato correttamente a una "Categoria" (es. SUV, Utilitaria, Furgone) per permetterne la ricerca da parte dei clienti (UC9).

- I veicoli devono essere registrati con i loro dati tecnici precisi (marca, modello, anno, cilindrata, posti) e assegnati immediatamente a un punto di noleggio (sede) fisico.

Soluzioni:

- Implementazione di un controller dedicato alla gestione del catalogo (CatalogoController o GestioneMezziController), che si occupa di ricevere in input i dati tecnici e di validarne la correttezza strutturale.
- Utilizzo del concetto di composizione/aggregazione: la creazione di un oggetto Mezzo fisico viene separata dalla sua DescrizioneMezzo (che contiene i dati tecnici) e dal suo TipoMezzo (che definisce le regole di categoria).
- Salvataggio dell'entità all'interno del CatalogoMezzi con lo stato iniziale impostato di default su DISPONIBILE.

MODELLO DELLE CLASSI

Relativamente al caso d'uso di inserimento di un nuovo mezzo, la progettazione ha portato all'individuazione delle seguenti classi concettuali:

- **CatalogoController:** Controller (<<control>>) che gestisce la logica di inserimento. Intercetta i dati immessi dall'operatore e orchestra la creazione degli oggetti di dominio.
- **MezzoService:** Classe di servizio che incapsula la logica di business core relativa ai mezzi. Riceve la richiesta dal controller, interroga il CatalogoMezzi per l'effettivo filtraggio e coordina il calcolo del preventivo, alleggerendo il controller dalle operazioni complesse.
- **CatalogoMezzi:** Repository che mantiene l'elenco in memoria (tramite HashMap) di tutti i veicoli della flotta. Espone il metodo aggiungiMezzo().
- **Mezzo:** Entità principale che rappresenta il singolo veicolo fisico (identificato da un ID univoco o targa). Contiene un riferimento al suo stato attuale (StatoMezzo) e alla sede di appartenenza.
- **DescrizioneMezzo:** Classe che incapsula i dettagli tecnici del veicolo (marca, modello, anno di immatricolazione, numero di posti).
- **TipoMezzo:** Entità che definisce la categoria del veicolo (necessaria per i filtri di ricerca e per le tariffe base).

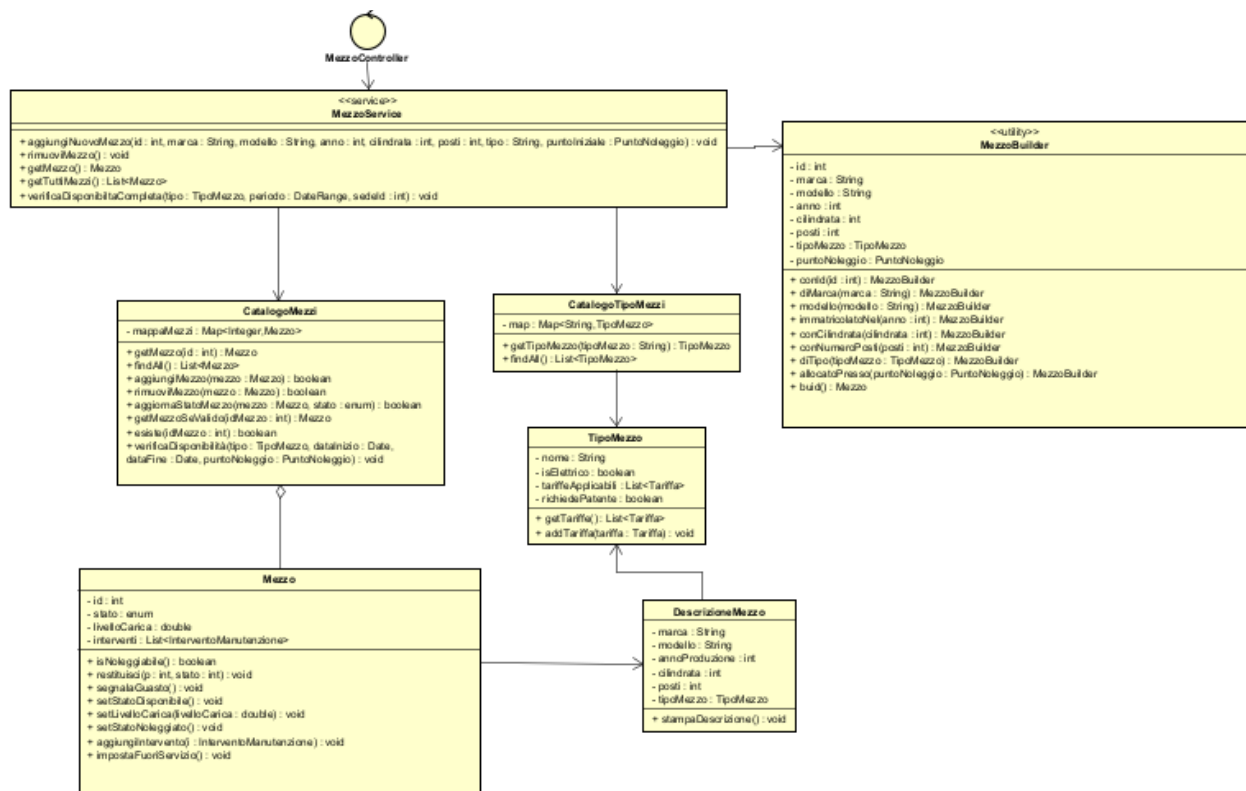
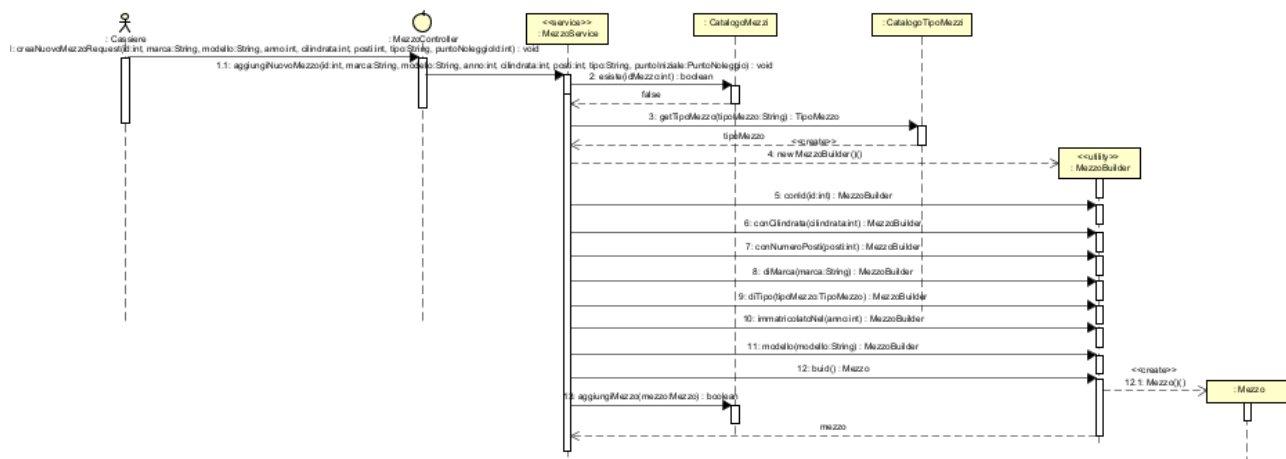


DIAGRAMMA DI SEQUENZA E CONTRATTI DELLE OPERAZIONI

Il diagramma di sequenza illustra l'interazione tra l'Amministratore/Operatore e il sistema. Mostra la creazione progressiva dei vari oggetti (Descrizione, Tipo, Mezzo) e la loro persistenza nel Catalogo.



Di seguito viene specificato formalmente il contratto per l'operazione di sistema principale.

CONTRATTO CO1: registraNuovoMezzo

- **OPERAZIONE:** registraNuovoMezzo (marca: String, modello: String, anno: int, cilindrata: int, posti: int, nomeTipo: String, idPuntoNoleggio: int)
- **RIFERIMENTI:** Aggiungi Nuovo Mezzo
- **PRECONDIZIONI:** * Il sistema è in funzione.
 - L'Operatore (o Amministratore) ha i privilegi necessari per modificare il catalogo ed è autenticato.
 - La categoria (nomeTipo) e la sede (idPuntoNoleggio) passate come parametri esistono già nel sistema.
- **POST CONDIZIONI:**
 - Il sistema recupera l'oggetto TipoMezzo e il PuntoNoleggio corrispondenti ai parametri forniti.
 - Viene creata una nuova istanza di DescrizioneMezzo contenente marca, modello, anno, cilindrata e posti, associandola al TipoMezzo (freccia <<create>>).
 - Viene istanziato un nuovo oggetto Mezzo (generando un nuovo ID univoco) a cui viene associata la DescrizioneMezzo.
 - Lo stato del Mezzo viene inizializzato a StatoMezzo.DISPONIBILE.
 - Il Mezzo viene associato al PuntoNoleggio specificato.
 - Il controller invoca aggiungiMezzo(mezzo) sul CatalogoMezzi per storicizzarlo nella mappa di sistema.
 - L'operazione restituisce un messaggio di conferma con l'ID assegnato al nuovo veicolo.

ITERAZIONE 3

UC6: Segnala Guasto

Problematiche:

- Durante un noleggio, o al momento della riconsegna, il cliente o l'operatore potrebbero riscontrare un malfunzionamento, un guasto meccanico o un danno estetico al veicolo.
- Il sistema deve reagire immediatamente rimuovendo il veicolo dalla flotta attiva (per i successivi noleggi o prenotazioni), al fine di evitare che un altro cliente prenoti un mezzo non sicuro o non marciante.
- È necessario tenere traccia della descrizione del problema per fornire ai meccanici le informazioni necessarie per la futura riparazione.

Soluzioni:

- Implementazione di un modulo per la segnalazione dei guasti gestito da un GuastoController.

- Creazione di un'entità Segnalazione (o TicketGuasto) che memorizza la data, il veicolo coinvolto e la descrizione del problema rilevato.
- Interazione immediata con il CatalogoMezzi per forzare l'aggiornamento dello stato del mezzo in IN_MANUTENZIONE, inibendo così le ricerche di disponibilità (UC9) per quello specifico ID.

MODELLO DELLE CLASSI

Relativamente al caso d'uso di segnalazione di un guasto, la progettazione ha portato all'individuazione delle seguenti classi concettuali:

- **NoleggioController:** Controller (<<control>>) responsabile della ricezione della segnalazione (da parte dell'operatore o del cliente) e del coordinamento tra l'aggiornamento dello stato del mezzo e il salvataggio del report.
- **NoleggioService:** Classe di servizio che incapsula la logica di business. È responsabile di coordinare le verifiche (es. controllare se il cliente è affidabile e se il mezzo è disponibile), istanziare la nuova entità Noleggio con i dati di partenza e richiamare i repository per il salvataggio.
- **Segnalazione:** Entità che modella il report del danno. Contiene l'ID del mezzo, la data della segnalazione, la descrizione testuale del guasto e l'utente che lo ha segnalato.
- **RegistroNoleggi:** Contiene il noleggio e il mezzo relativi alla segnalazione
- **Mezzo:** Entità il cui attributo stato viene modificato.

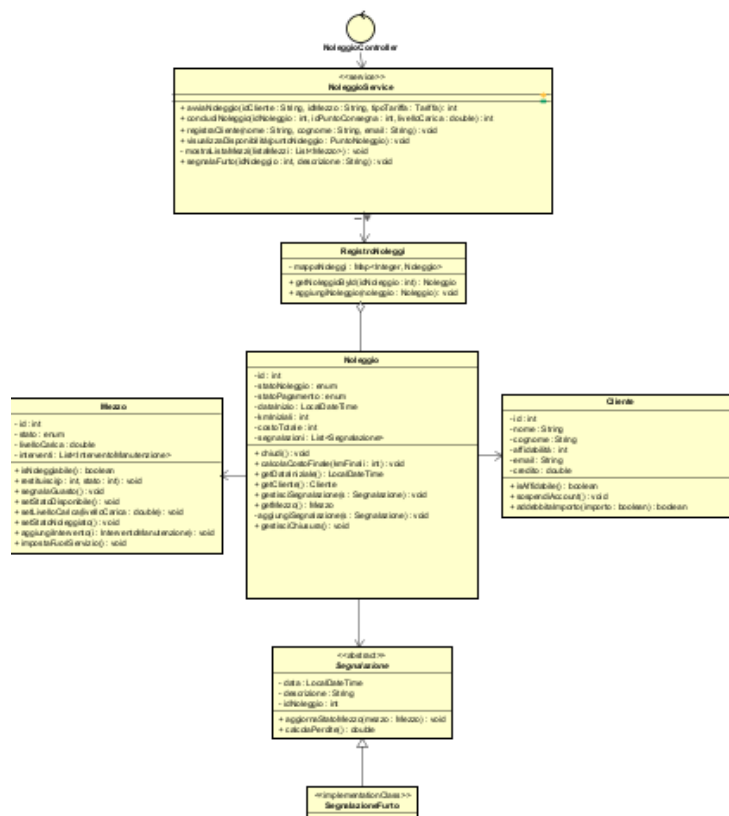
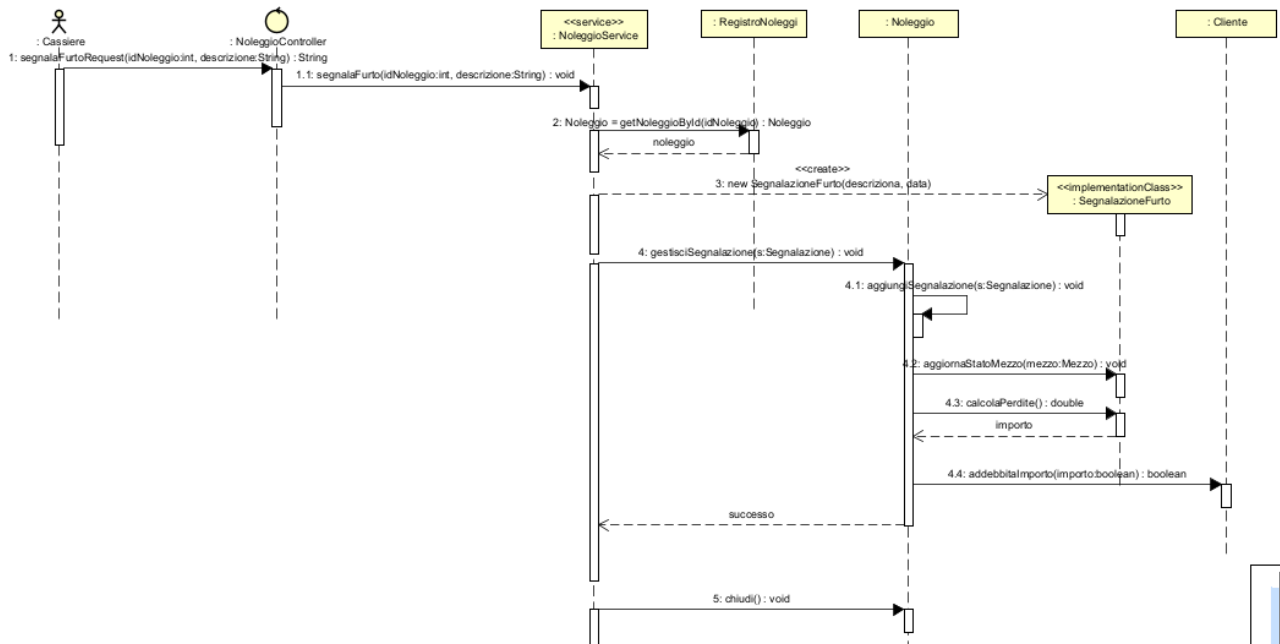


DIAGRAMMA DI SEQUENZA E CONTRATTI DELLE OPERAZIONI

Il diagramma di sequenza illustra l'interazione per l'apertura di un ticket di guasto. Evidenzia la creazione del report e il fondamentale cambio di stato del veicolo all'interno del catalogo.



Di seguito viene specificato formalmente il contratto per l'operazione di sistema principale.

CONTRATTO CO1: segnalaGuasto

- **OPERAZIONE:** segnalaGuasto (idMezzo: int, descrizione: String, idSegnalatore: int)
- **RIFERIMENTI:** Segnala Guasto
- **PRECONDIZIONI:** * Il sistema è in funzione.
 - Il veicolo identificato da idMezzo esiste nel CatalogoMezzi.
 - (Opzionale) Il veicolo si trova attualmente in stato NOLEGGIATO o DISPONIBILE.
- **POST CONDIZIONI:**
 - Viene istanziato un nuovo oggetto Segnalazione contenente la descrizione del problema e associato al mezzo (freccia <<create>>).
 - Il sistema salva la Segnalazione all'interno dell'apposito registro in memoria.
 - Il sistema invoca l'aggiornamento sul CatalogoMezzi, modificando lo stato del Mezzo da quello attuale a IN_MANUTENZIONE.
 - Il sistema restituisce un messaggio di conferma della presa in carico del problema.

UC8: Effettua Manutenzione

Problematiche:

- A seguito di un guasto o di un controllo periodico, i veicoli necessitano di interventi di riparazione. Durante questo periodo, non devono essere disponibili per il noleggio.
- L'officina o l'operatore deve poter registrare l'avvenuta riparazione, indicando i dettagli dell'intervento (descrizione e costo).
- Il sistema deve poter gestire in modo robusto gli errori operativi, come il tentativo di riparare un mezzo non esistente nel database o un mezzo che non si trova nello stato corretto per la manutenzione.
- Al termine dell'operazione, il veicolo deve essere ripristinato a uno stato operativo (es. DISPONIBILE) oppure dismesso definitivamente (es. FUORI_SERVIZIO).

Soluzioni:

- Implementazione di un `ManutenzioneController` che funge da intermediario tra l'interfaccia dell'operatore e il livello di persistenza.
- Utilizzo del `CatalogoMezzi` per recuperare il veicolo tramite ID. Per garantire la robustezza, sono state implementate eccezioni personalizzate (es. `EnitaNonTrovataException`) che interrompono il flusso in caso di anomalie, avvisando l'operatore.
- Storizzazione dell'intervento tramite la creazione di un oggetto `InterventoManutenzione` associato alla storia del veicolo, e contestuale aggiornamento dello `StatoMezzo`.

MODELLO DELLE CLASSI

Relativamente al caso d'uso di esecuzione della manutenzione, la progettazione ha portato all'individuazione delle seguenti classi concettuali:

- **ManutenzioneController:** Controller (<<control>>) responsabile della gestione del flusso di riparazione. Riceve l'ID del veicolo e i dati dell'intervento, coordinando la validazione e l'aggiornamento.
- **ManutenzioneService:** Classe di servizio che incapsula la logica di business relativa alle riparazioni. Coordina la validazione del veicolo, la creazione del nuovo `InterventoManutenzione` e l'aggiornamento dello stato del mezzo. È responsabile del lancio delle eccezioni custom qualora i controlli di validità falliscano.
- **CatalogoMezzi:** Repository che contiene la logica di ricerca sicura (`getMezzoSeValido`) e di aggiornamento di stato (`aggiornaStatoMezzo`).
- **Mezzo:** Entità su cui viene effettuata la manutenzione. Contiene una lista storica dei propri interventi (`List<InterventoManutenzione>`) e il proprio stato corrente.
- **InterventoManutenzione:** Entità che modella la singola ricevuta/scheda di riparazione (data, descrizione, costo, operatore che l'ha eseguita).

- **Eccezioni Custom:** Classi come `EnitaNonTrovataException` e `StatoNonValidoException` che estendono `RuntimeException` per gestire i flussi alternativi (Scenari di Errore).

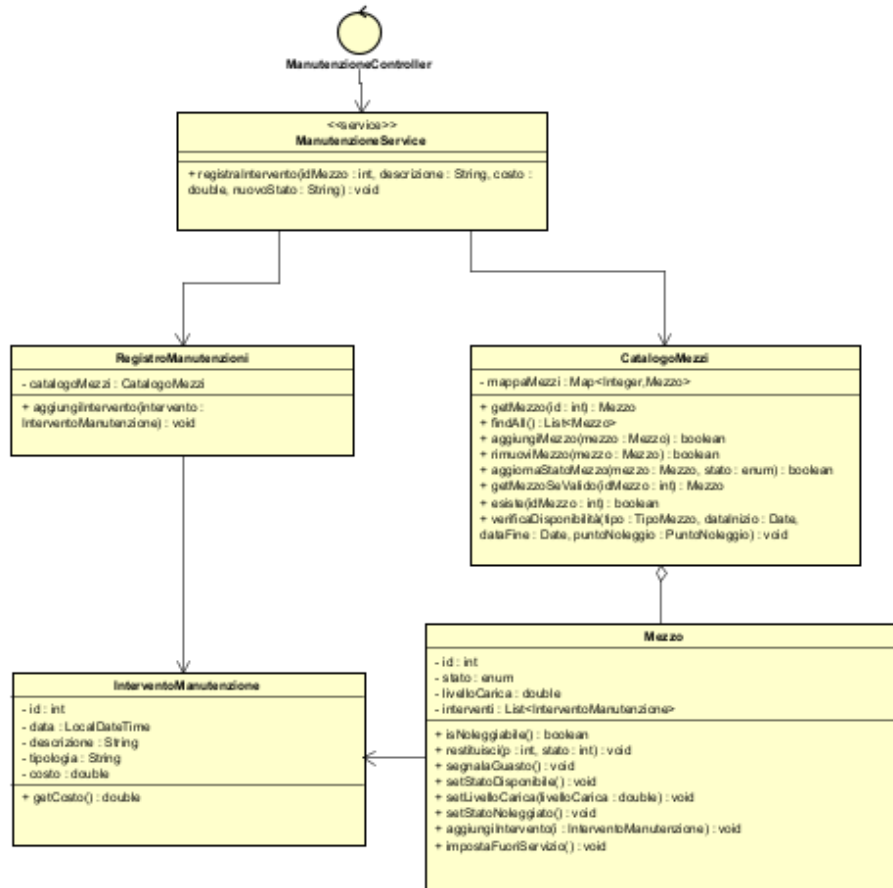
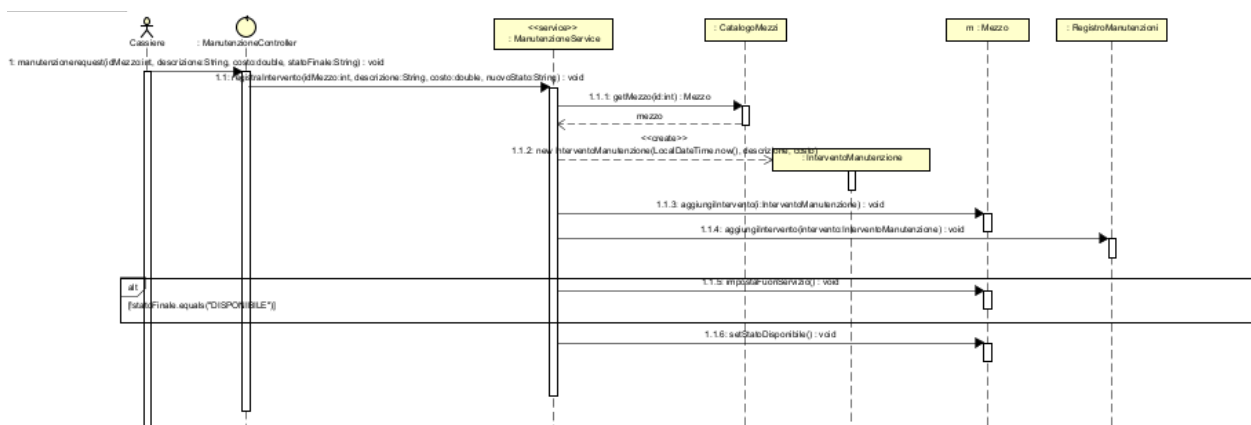


DIAGRAMMA DI SEQUENZA E CONTRATTI DELLE OPERAZIONI

Il diagramma di sequenza mostra l'interazione tra il Meccanico/Operatore e il sistema. Evidenzia la fase di validazione del veicolo (con possibile lancio di eccezioni) e la successiva creazione del record di manutenzione.



Di seguito viene specificato formalmente il contratto per l'operazione di sistema principale.

CONTRATTO CO1: registaIntervento

- **OPERAZIONE:** registaIntervento (idMezzo: int, descrizione: String, costo: double, nuovoStato: StatoMezzo)
- **RIFERIMENTI:** Effettua Manutenzione (UC8)
- **PRECONDIZIONI:** * Il sistema è in funzione.
 - Il Meccanico (o Operatore) è autenticato.
 - Il Mezzo identificato da idMezzo è presente nel CatalogoMezzi e il suo stato attuale è IN_MANUTENZIONE.
- **POST CONDIZIONI:**
 - Il sistema invoca getMezzo(idMezzo) sul CatalogoMezzi. Se non trovato, l'operazione fallisce sollevando EnitaNonTrovataException.
 - Viene istanziato un nuovo oggetto InterventoManutenzione con i dettagli forniti (freccia <<create>>).
 - L'oggetto InterventoManutenzione viene aggiunto alla lista storica (storicoInterventi) dell'istanza Mezzo corrispondente.
 - Lo stato dell'oggetto Mezzo viene modificato al valore specificato nel parametro nuovoStato (es. passa da IN_MANUTENZIONE a DISPONIBILE).
 - L'operazione restituisce conferma del corretto salvataggio a video.

UC9: Effettua Prenotazione WEB

Problematiche:

- Dopo aver visualizzato i veicoli disponibili, il cliente ha bisogno di bloccare il mezzo scelto confermando le date e la sede di ritiro.
- È fondamentale evitare l'overbooking: nel momento esatto in cui la prenotazione viene confermata, il sistema deve rimuovere il mezzo dalla lista dei veicoli disponibili per quelle date.
- Il sistema deve generare un codice identificativo univoco (PNR) da fornire al cliente come ricevuta.
- Il calcolo del preventivo finale deve essere flessibile, basandosi su regole tariffarie che potrebbero cambiare nel tempo (es. tariffa oraria vs giornaliera).

Soluzioni:

- Implementazione del metodo creaNuovaPrenotazione all'interno del PrenotazioneController per orchestrare il salvataggio dei dati.
- Creazione dell'entità Prenotazione che gestisce autonomamente la generazione di un codice PNR alfanumerico (utilizzando la classe UUID di Java).

- **Applicazione del Pattern Strategy:** L'entità Prenotazione utilizza l'interfaccia ITariffa per delegare il calcolo del costoTotale in base al DateRange selezionato, separando così la logica di business commerciale dalla pura struttura dati.
- Utilizzo del RegistroPrenotazioni per la persistenza in memoria della transazione e chiamata coordinata al CatalogoMezzi per aggiornare lo stato del veicolo in NOLEGGIATO (o PRENOTATO).

MODELLO DELLE CLASSI

Relativamente al caso d'uso di conferma della prenotazione web, la progettazione ha portato all'individuazione delle seguenti classi concettuali:

- **PrenotazioneController:** Controller (<<control>>) responsabile della gestione del flusso finale. Riceve l'input di conferma e coordina le entità e i repository.
- **Prenotazione:** Entità centrale del caso d'uso. Contiene i riferimenti a Cliente, Mezzo, PuntoNoleggio e DateRange. Espone il metodo privato calcolaCostoStimato() che sfrutta il pattern Strategy, e generaPNR().
- **RegistroPrenotazioni:** Repository (implementato come Singleton) che mantiene la lista di tutte le prenotazioni attive nel sistema.
- **CatalogoMezzi:** Repository a cui viene delegato il cambio di stato del veicolo per inibirlo dalle ricerche future.
- **ITariffa (Pattern Strategy):** Interfaccia implementata da classi concrete (es. TariffaGiornaliera) che ricevono i minuti di durata del noleggio calcolati dalla Prenotazione tramite la libreria ChronoUnit di Java.

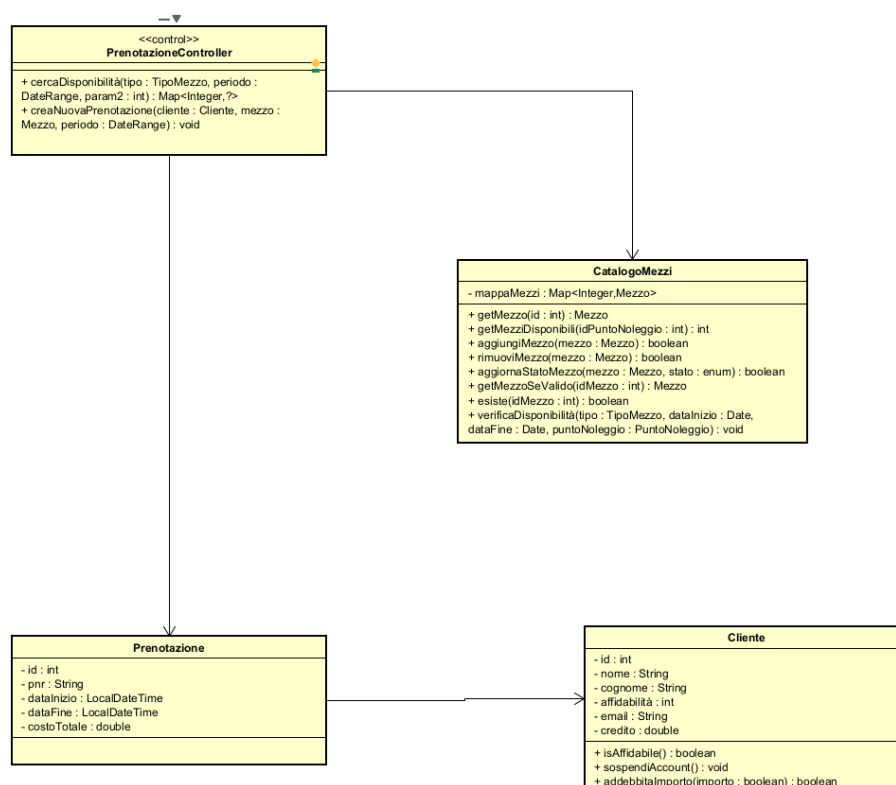
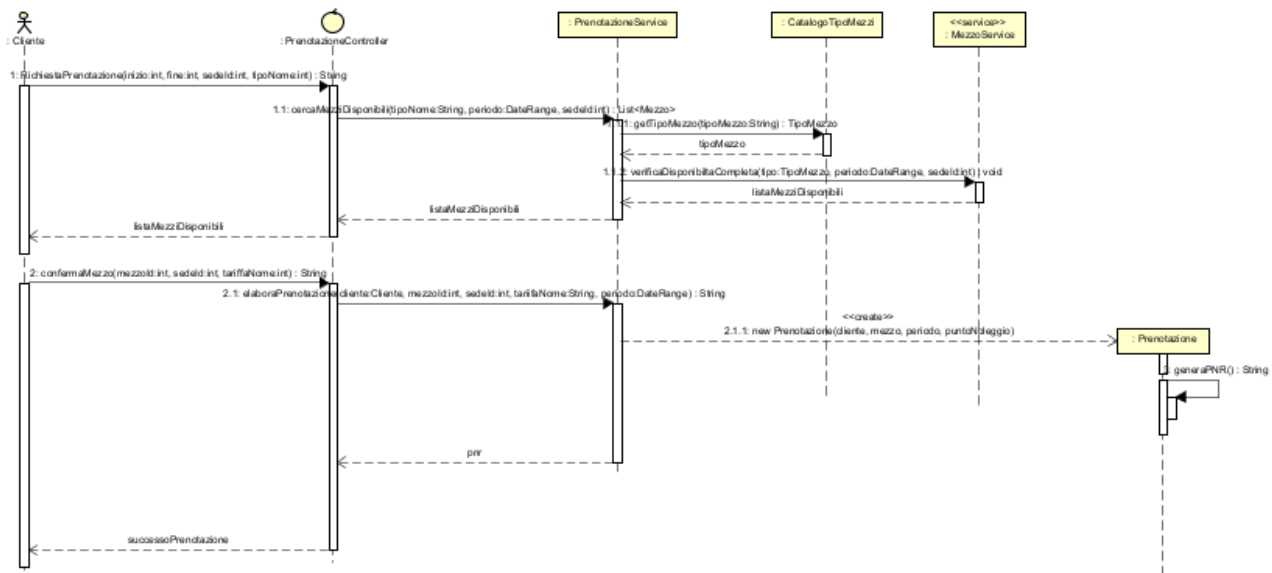


DIAGRAMMA DI SEQUENZA E CONTRATTI DELLE OPERAZIONI

Il diagramma di sequenza mostra l'interazione finale tra il Cliente e il sistema. Evidenzia la creazione della prenotazione, i "self-message" per il calcolo del costo e la generazione del PNR, il salvataggio nel registro e l'aggiornamento di stato.



Di seguito viene specificato formalmente il contratto per l'operazione di sistema principale.

CONTRATTO CO1: creaNuovaPrenotazione

- **OPERAZIONE:** creaNuovaPrenotazione (cliente: Cliente, mezzo: Mezzo, periodo: DateRange, sede: PuntoNoleggio)
- **RIFERIMENTI:** Effettua Prenotazione Web (UC9)
- **PRECONDIZIONI:** * Il sistema è in funzione.
 - Il Cliente ha effettuato l'accesso con successo.
 - L'oggetto mezzo è presente nel CatalogoMezzi e il suo stato è verificato come DISPONIBILE.
- **POST CONDIZIONI:**
 - Viene istanziato un nuovo oggetto Prenotazione (freccia <<create>>).
 - Il sistema auto-genera e assegna un codice PNR univoco di 6 caratteri alla prenotazione.
 - Il sistema delega il calcolo matematico del costo all'implementazione attiva di ITariffa (es. TariffaGiornaliera) e lo memorizza nell'attributo costoTotale.
 - La prenotazione viene inserita all'interno della mappa nel RegistroPrenotazioni.

- Il sistema invoca `aggiornaStatoMezzo(mezzo, StatoMezzo.NOLEGGIATO)` sul `CatalogoMezzi` per bloccare il veicolo.
- L'operazione restituisce la stringa contenente il codice PNR generato per mostrarlo a video al cliente.

UC10: Gestisci Blacklist

Problematiche:

- L'azienda di noleggio deve potersi tutelare da clienti che si sono dimostrati inaffidabili (es. insolvenza nei pagamenti, danni gravi ai veicoli, ritardi estremi nella riconsegna).
- È necessario un meccanismo per inibire l'accesso ai servizi: un cliente segnalato non deve più poter visualizzare i preventivi o confermare nuove prenotazioni (UC9) finché la sua posizione non viene regolarizzata.
- Deve essere mantenuta traccia della motivazione per cui il cliente è stato sospeso, per permettere allo staff amministrativo di gestire eventuali ricorsi o sblocchi futuri.

Soluzioni:

- Implementazione di un modulo di gestione clienti (`GestioneClientiController` o `BlacklistController`) accessibile solo al personale autorizzato (Amministratore o Manager).
- Introduzione di un attributo di stato all'interno dell'entità Cliente (es. una variabile booleana `isAffidabile` o un Enum `StatoAccount.SOSPESO`).
- Inserimento di un blocco logico (guardia) nei flussi di "Avvia Noleggio" e "Effettua Prenotazione", che verifica l'affidabilità del cliente prima di procedere, lanciando un'eccezione dedicata (es. `ClienteSospesoException`) in caso di esito negativo.

MODELLO DELLE CLASSI

Relativamente al caso d'uso di gestione della blacklist, la progettazione ha portato all'individuazione delle seguenti classi concettuali:

- **BlacklistController (o GestioneClientiController):** Controller (`<<control>>`) che gestisce la richiesta di sospensione. Riceve l'ID del cliente e la motivazione dall'operatore.
- **ClienteService:** Classe di servizio che incapsula la logica di business relativa alla gestione degli account. Riceve l'ID e la motivazione dal controller, interroga il `RegistroClienti` per recuperare il profilo utente e applica le regole di business per la sospensione, gestendo eventuali eccezioni (es. `EntitaNonTrovataException` se il cliente non esiste).
- **RegistroClienti:** Repository in memoria che contiene la mappa di tutti i clienti registrati. Fornisce il metodo sicuro per recuperare l'istanza del cliente verificandone l'esistenza.

- **Cliente:** Entità di dominio che subisce l'aggiornamento. Espone metodi come `spendiAccount(motivazione)` o `setAffidabile(false)`. Può contenere una lista di note disciplinari o un attributo `motivazioneSospensione`.

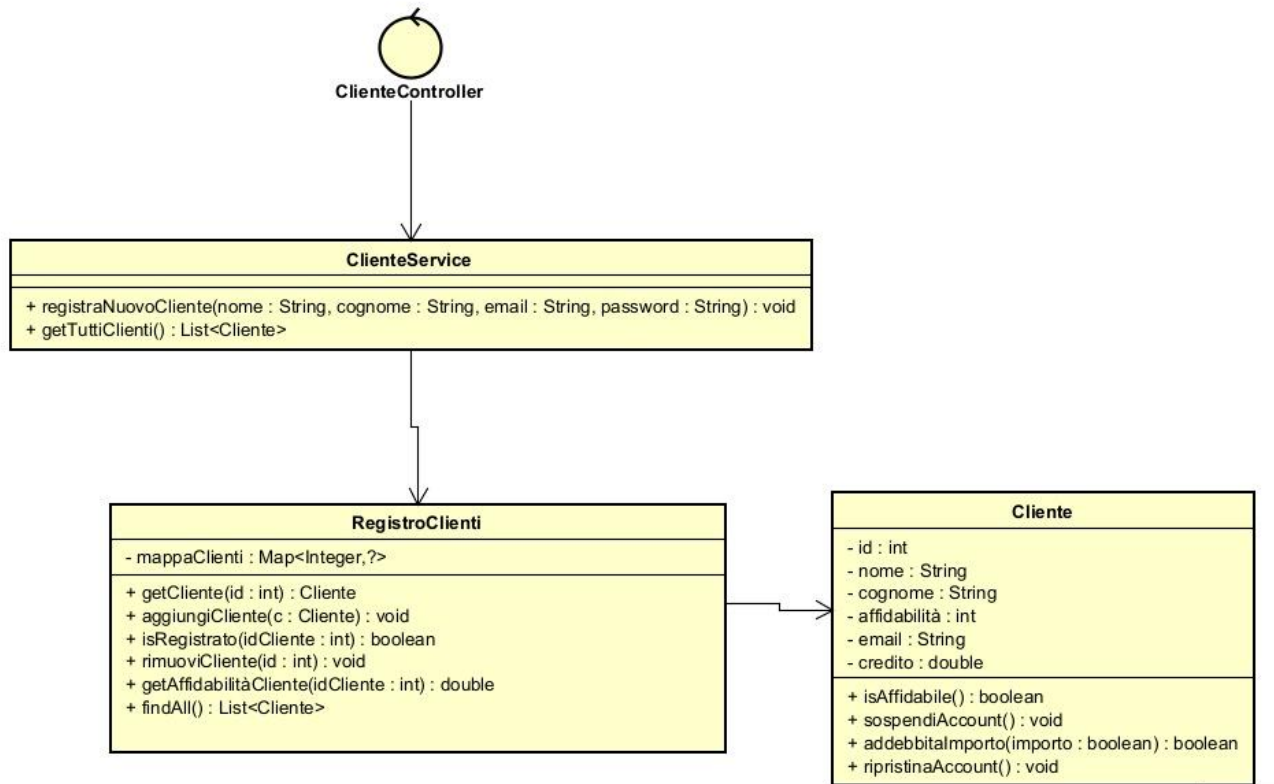
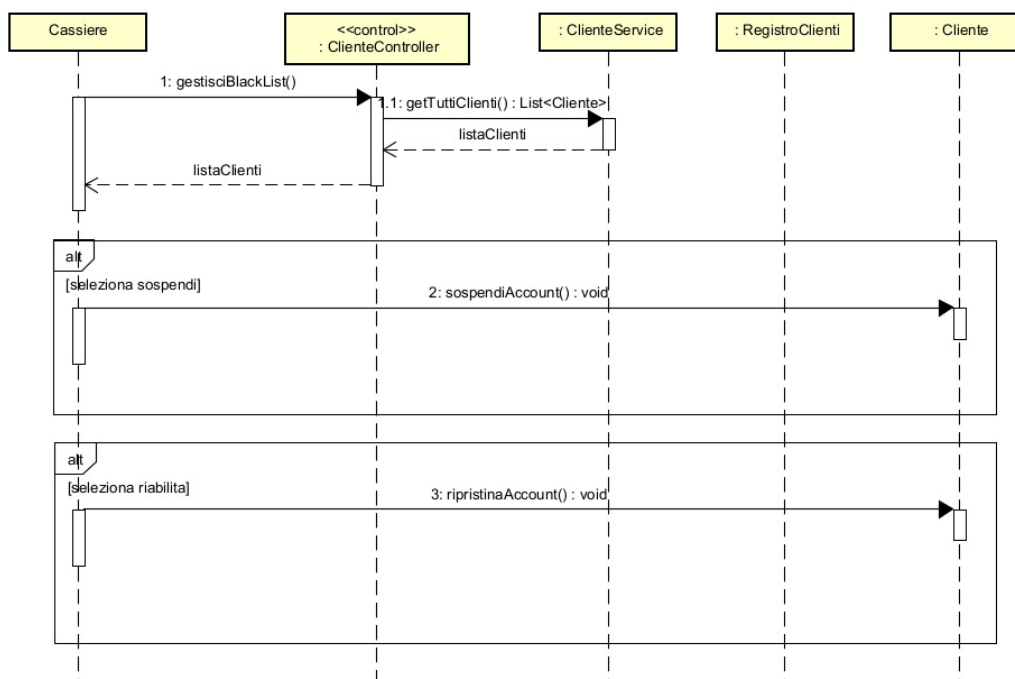


DIAGRAMMA DI SEQUENZA E CONTRATTI DELLE OPERAZIONI

Il diagramma di sequenza illustra l'interazione tra l'Amministratore e il sistema. Mostra la ricerca del cliente nel registro e la successiva mutazione del suo stato interno.



Di seguito viene specificato formalmente il contratto per l'operazione di sistema principale.

CONTRATTO CO1: inserisciInBlacklist

- **OPERAZIONE:** inserisciInBlacklist (idCliente: int, motivazione: String)
- **RIFERIMENTI:** Gestisci Blacklist
- **PRECONDIZIONI:** * Il sistema è in funzione.
 - L'utente corrente è autenticato e possiede i privilegi di Amministratore o Manager.
 - Il Cliente identificato da idCliente esiste nel RegistroClienti e si trova attualmente in uno stato attivo (non è già sospeso).
- **POST CONDIZIONI:**
 - Il sistema invoca getClienti(idCliente) sul RegistroClienti. Se il cliente non esiste, solleva un'eccezione (EntitaNonTrovataException).
 - Il sistema modifica lo stato dell'oggetto Cliente impostando l'attributo di affidabilità a false (o lo stato a SOSPESO).
 - La stringa motivazione viene salvata all'interno del profilo del Cliente per future consultazioni.
 - Le successive richieste di prenotazione (UC9) per questo specifico Cliente verranno automaticamente respinte dal sistema.
 - L'operazione restituisce a video un messaggio di conferma dell'avvenuta sospensione.

DESIGN PATTERN UTILIZZATI

Per garantire che l'architettura del sistema sia scalabile, mantenibile e flessibile ai futuri cambiamenti dei requisiti, la progettazione orientata agli oggetti ha fatto largo uso dei Design Pattern consolidati (GoF). L'adozione di tali pattern ha permesso di risolvere problemi ricorrenti di design, riducendo l'accoppiamento tra le classi e aumentando la coesione.

Di seguito vengono analizzati i principali pattern implementati nel dominio applicativo:

1. Pattern Strategy (Calcolo della Tariffa Base)

- **Problema:** Il costo di un noleggio può essere calcolato seguendo algoritmi molto diversi tra loro (es. a tariffazione oraria, a tariffazione giornaliera, a chilometraggio, ecc.). Gestire queste casistiche con costrutti condizionali (come grandi blocchi if/else o switch) all'interno della classe Noleggio avrebbe violato il principio Open/Closed (OCP).

- **Soluzione:** È stato adottato il pattern comportamentale **Strategy**. È stata definita un'interfaccia comune `ITariffa` che espone il metodo per il calcolo del costo. Le specifiche logiche di calcolo sono state incapsulate in classi concrete separate (es. `TariffaOraria`, `TariffaGiornaliera`). Il sistema a runtime inietta la strategia corretta all'interno del noleggio, permettendo all'algoritmo di variare in modo totalmente trasparente per il client.

2. Pattern Decorator (Aggiunta degli Extra alla Tariffa)

- **Problema:** Oltre alla tariffa base, i clienti possono selezionare una serie di opzioni aggiuntive (es. assicurazione furto/incendio, seggiolino per bambini) oppure incorrere in maggiorazioni contrattuali (es. penale per giovane guidatore). Creare una sottoclasse per ogni possibile combinazione di tariffa base ed extra avrebbe portato a un'esplosione combinatoria delle classi (es. `TariffaOrariaConAssicurazione`, `TariffaGiornalieraConAssicurazioneEGiovaneGuidatore`).
- **Soluzione:** Si è ricorsi al pattern strutturale **Decorator**. Tramite una classe astratta `TariffaDecorator` che implementa la stessa interfaccia `ITariffa`, è possibile "avvolgere" (wrap) l'oggetto tariffa originale. Classi concrete come `AssicurazioneFurto` o `PenaleGiovaneGuidatore` aggiungono dinamicamente il proprio costo a quello dell'oggetto decorato. In questo modo, è possibile impilare infiniti extra a runtime in modo flessibile.

3. Pattern Builder (Creazione della Classe Complessa Mezzo)

- **Problema:** La classe `Mezzo` (e la sua componente interna `DescrizioneMezzo`) richiede un numero elevato di parametri per essere istanziata correttamente (marca, modello, anno di immatricolazione, cilindrata, posti, tipo, sede di allocazione). L'utilizzo di un costruttore tradizionale avrebbe generato un *Anti-Pattern*, rendendo il codice prone a errori (es. invertire l'anno con la cilindrata) e difficile da leggere.
- **Soluzione:** L'istanziamento è stato demandato al pattern creazionale **Builder**. La classe `MezzoBuilder` fornisce un'interfaccia fluente (*fluent interface*) che permette di configurare il veicolo passo dopo passo (es. `.diMarca("Fiat").modello("Panda")`). Il pattern ritarda la creazione dell'oggetto finale fino alla chiamata del metodo `build()`, all'interno del quale vengono concentrate tutte le validazioni formali (es. obbligatorietà della marca o validità dell'anno di immatricolazione), impedendo la creazione di veicoli in uno stato inconsistente.

4. Pattern State (Gestione dello Stato del Mezzo)

- **Problema:** Il ciclo di vita di un veicolo è complesso e fortemente vincolato dalla realtà fisica. Un veicolo può essere noleggiato solo se è *Disponibile*, non può essere mandato in officina se è già *Noleggiato*, e non può essere prenotato se è *Fuori Servizio* o *In Manutenzione*. Affidare il controllo di queste regole a una singola variabile di stato (`String` o `Enum`) avrebbe riempito i metodi della classe `Mezzo` di complesse logiche condizionali, difficili da testare ed espandere.
- **Soluzione:** È stato implementato il pattern comportamentale **State**. L'interfaccia `IStatoMezzo` definisce le azioni possibili (es. `noleggia()`, `restituisci()`, `inviaInManutenzione()`). Gli stati concreti (es. `StatoDisponibile`, `StatoNoleggiato`,

StatoInManutenzione) implementano queste interazioni. La classe Mezzo delega l'esecuzione dei comandi al suo stato corrente. Se un'azione non è permessa nello stato attuale (es. noleggiare un'auto in officina), la specifica classe di stato solleva autonomamente una StatoNonValidoException. Questo approccio ha reso la macchina a stati finiti del veicolo intrinsecamente sicura e perfettamente allineata al dominio reale.

INTERFACCIA UTENTE WEB

L'interfaccia utente del sistema di noleggio è stata realizzata come applicazione web basata su Spring Boot, seguendo il pattern architetturale Model-View-Controller (MVC). L'applicazione viene avviata tramite la classe App, che espone il sistema all'indirizzo <http://localhost:8080/dashboard>. All'avvio, il sistema inizializza automaticamente dati di test tramite un CommandLineRunner, permettendo l'utilizzo immediato dell'applicazione in ambiente di sviluppo.

Architettura della Presentazione

L'interfaccia segue una chiara separazione dei livelli:

- View (Frontend web): responsabile della presentazione delle pagine e della raccolta degli input utente.
- Controller Spring: gestiscono le richieste HTTP e coordinano i servizi applicativi.
- Service Layer: contiene la logica di business.
- Repository Layer: gestisce la persistenza in memoria dei dati.

Questa separazione garantisce basso accoppiamento, alta manutenibilità e facilità di testing.

Flusso di Avvio dell'Applicazione

All'avvio dell'applicazione Spring Boot esegue la classe App, registra il bean CommandLineRunner e invoca il metodo di inizializzazione dei dati, caricando automaticamente tipi mezzo, punti noleggio, clienti, mezzi, tariffe e cassiere amministratore. Ciò consente di avere un ambiente immediatamente operativo per demo e test.

Dashboard Operativa

La dashboard rappresenta il punto di ingresso principale per gli operatori e consente l'accesso alle funzionalità core del sistema, tra cui visualizzazione disponibilità mezzi, gestione clienti, avvio e chiusura noleggi, prenotazioni web e gestione manutenzioni.

Tecnologie Utilizzate

Nel progetto sono state utilizzate le seguenti tecnologie per il layer di presentazione: Spring Boot, Spring MVC ed eventuale template engine per la generazione delle pagine web.

TESTING

La validazione del sistema "Noleggio Mezzi" è stata effettuata adottando una strategia di Unit e Integration Testing basata su JUnit 5.

L'obiettivo principale della fase di testing è stato quello di isolare e verificare la **Business Logic** all'interno del *Service Layer*, garantendo che i controlli di sicurezza, le transizioni di stato del *Pattern State* e le interazioni con i Repository in memoria operassero in conformità con i requisiti dei Casi d'Uso (UC).

Di seguito è riportata la documentazione delle suite di test implementate per i componenti core del sistema.

1. PrenotazioneServiceTest (UC9 - Effettua Prenotazione)

Motivazione: Questo componente rappresenta il motore principale del sistema lato utente. Era fondamentale testare l'algoritmo di incrocio delle date per evitare la sovrapposizione di prenotazioni sullo stesso veicolo e garantire che solo i clienti in regola potessero accedere al servizio.

- **Scenario di Successo (Happy Path):**
 - *Obiettivo:* Verificare che, richiedendo un veicolo in un periodo libero da impegni, il sistema lo restituisca correttamente nella lista delle disponibilità.
- **Scenari Limite e di Fallimento (Error Path):**
 - *Sovrapposizione Date:* Si è simulata la ricerca di un veicolo in date parzialmente coincidenti con una prenotazione già esistente. Il test garantisce che il veicolo venga correttamente omesso (nascosto) dai risultati di ricerca.
 - *Cliente Sospeso:* Si è verificato che un cliente con account sospeso (affidabilità azzerata) venga respinto durante la fase di conferma, sollevando un'apposita eccezione (*StatoNonValidoException*) e impedendo il salvataggio della prenotazione.

2. NoleggioServiceTest (Avvia e Concludi Noleggio / UC4 e UC5)

Motivazione: Il servizio di noleggio gestisce le operazioni fisiche al banco e i pagamenti. Il focus dei test è stato verificare la robustezza del *Pattern State* e la corretta gestione del ciclo di vita del noleggio (apertura e chiusura).

- **Scenari di Successo:**
 - *Avvio Noleggio:* Assicura che, forniti dati validi, il noleggio venga creato, salvato e che lo stato del veicolo passi correttamente a NOLEGGIATO.
 - *Chiusura Noleggio:* Verifica che, alla restituzione, il mezzo torni allo stato DISPONIBILE, il noleggio risulti chiuso e il credito del cliente venga scalato per coprire il costo del servizio.
- **Scenari di Fallimento e Anomalie:**
 - *Avvio con Batteria Scarica:* Garantisce che il sistema rifiuti categoricamente di avviare il noleggio se il livello di carica del veicolo è inferiore alla soglia di sicurezza.
 - *Credito Insufficiente al Rientro:* Verifica che, se il cliente non dispone dei fondi necessari per saldare la fattura, il sistema sollevi una *PagamentoException*, sospenda preventivamente l'account del cliente, ma rilasci comunque il veicolo per renderlo nuovamente disponibile alla flotta.

3. ManutenzioneServiceTest (UC8 - Gestisci Manutenzione)

Motivazione: I test per questo servizio avevano lo scopo di validare le transizioni più complesse del *Pattern State*, assicurandosi che un veicolo guasto non potesse mai essere accidentalmente noleggiato e che il registro degli interventi (storico) venisse mantenuto coerente.

- **Scenari di Successo e Transizioni:**

- *Invio in Officina:* Dimostra che il veicolo transita nello stato `IN_MANUTENZIONE`, scomparendo dalle disponibilità.
- *Ripristino (Rientro in Flotta):* Verifica che, terminata la riparazione, lo stato torni a `DISPONIBILE` e che l'intervento (costo e descrizione) venga correttamente tracciato nello storico del veicolo.
- *Dismissione (Fuori Servizio):* Assicura che, in caso di danni irreparabili, il veicolo transiti nello stato terminale `FUORI_SERVIZIO`, impedendone definitivamente il noleggio futuro.

4. MezzoServiceTest (Gestione Parco Auto)

Motivazione: Il test si è concentrato sulla validazione del pattern creazionale **Builder** (*MezzoBuilder*), incaricato di istanziare oggetti complessi garantendo l'integrità dei dati fin dalla loro creazione.

- **Scenario di Successo:**

- *Aggiunta Mezzo Valido:* Verifica che un mezzo inserito con parametri corretti venga persistito nel catalogo con lo stato iniziale impostato automaticamente su `DISPONIBILE`.

- **Scenari di Fallimento (Validazione Input):**

- *Dati Obbligatorie e Formali:* Il test dimostra che se vengono passati parametri nulli (es. marca mancante) o valori illogici (es. anno di immatricolazione antecedente al 1900), il Builder intercetta l'anomalia lanciando un'eccezione (`IllegalStateException`) ed evitando di inquinare il catalogo.

5. ClienteServiceTest (Gestione Utenti)

Motivazione: Questi test garantiscono l'integrità del database degli utenti e il corretto funzionamento delle logiche di sicurezza (come la validazione dell'email o il blocco di utenti duplicati).

- **Scenari di Successo:**

- *Registrazione Standard:* Verifica che un nuovo cliente venga registrato e risulti "affidabile" di default.
- *Sospensione e Riabilitazione:* Dimostra la corretta mutazione dello stato del profilo utente da parte dell'amministratore (`UC1/Blacklist`).

- **Scenari di Fallimento:**

Gestione Duplicati: Il test garantisce che un tentativo di registrazione con un'email già presente nel sistema venga respinto (`IllegalArgumentException`), tutelando il sistema da registrazioni ridondanti o malevole.